Where to Stop:
Tile-Adaptive Early Exit in a Learned Image Decoder

Anonymous CVPR submission · Paper ID ****

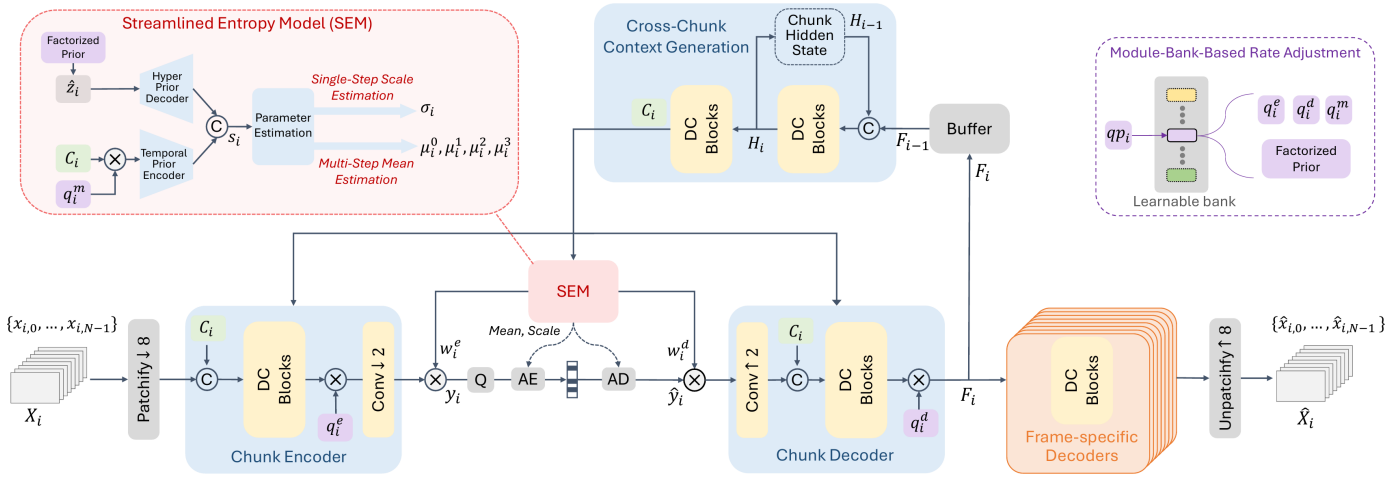

Figure 1. The decoder we modify. Figure 3 of DCVC-UF [14], reproduced. Everything up to the reconstruction stays frozen in this work: the patch embedding, the chunk encoder, the entropy model and the coded payload. FLEX-UF replaces the frame-specific decoders on the right with a ladder of exits taken per tile.

Abstract

A learned image decoder reconstructs a picture by running a fixed network over it, and the network is fixed by design: that is what lets any decoder read any file. On the intra decoder we study, that network costs 453 GMAC for a 1080p frame and spends the same arithmetic everywhere in it: a flat sky and a face are decoded at the same price. Making decoders cheaper is now its own line of work, and almost all of it changes the model, and so the bitstream: a file encoded yesterday cannot benefit. What is left to vary is not what the decoder is, but how much of it runs where.

Here we show that decoding computation can be allocated by content on a decoder that is not allowed to change. A frame is cut into tiles after the shared part of the decoder has run, and each tile leaves the remaining trunk at its own depth, with the encoder, the entropy model and the coded latent untouched. On 53 CTC intra frames a 0.1 dB constraint removes 29.2 to 15.2% of decoder multiply-accumulates across the rate range, 21.5% on average, for a BD-Rate cost of 1.03%.

Two findings are not specific to this decoder. Adaptivity is usable only inside a window that tiling itself creates, between a floor below which no allocation meets the budget and a saturation point above which none improves; rescaling the budget between those limits collapses five rate curves 14.0 points apart onto one within 1.7. And what is needed to allocate is already in the file: routing on the bits the entropy model has spent on a tile beats our trained 144 K-parameter router at every rate, by up to 3.4 points, while agreeing with the exhaustive search on fewer tiles. A compressed representation records not only what to reconstruct, but how much computation the reconstruction is worth.

1. Introduction

Learned codecs are compared on rate and distortion, with complexity given as a single number, so many GMAC per frame or so many milliseconds (Figure 3). That number is a constant, and it is constant for a good reason: a decoder that is the same everywhere is what lets a file encoded anywhere be read anywhere. A learned decoder runs the same graph on a page of text as on a cloudless sky, although the sky is much the easier picture (Figure 2).

Classification networks abandoned this a decade ago. Early-exit architectures attach classifiers at intermediate depths and stop as soon as the prediction is confident [3, 15, 20], and the literature around them is by now mature. Super-resolution adopted the same idea spatially. ClassSR [12] routes image patches to networks of different capacity by difficulty, and APE [1] exits patches at different depths of one network. Learned compression has taken a different route. Slimmable autoencoders [22, 23] give one model several complexity levels, but the level is chosen *per stream* rather than per region, and switching it changes the bitstream.

We ask the spatial-adaptivity question inside a learned decoder, and we keep the property that makes it worth having: **the encoder is frozen and the coded payload is unchanged.** Whatever we do has to consume the bitstream the released encoder already produces. That rules out re-training the analysis transform, changing the entropy model, or altering the latent. It leaves one place to spend adaptivity, the synthesis transform.

Our method, FLEX-UF, attaches exits at intervals through the twelve residual blocks (Figure 1) of the DCVC-UF intra decoder; we call that ordered set of exits the *ladder*. The first j blocks run over the whole frame. The rest run per tile, over a set of tiles that shrinks with depth, and the shrinkage is where the saving comes from. Each exit hands its feature to the shared head through a small pointwise adapter, zero-initialised so that at step zero the deepest exit reproduces the released decoder bit-exactly.

Getting this to work depended less on the ladder than on two problems that have little to do with early exit as it is usually studied.

Tiling has a large price.

Spatial adaptivity needs tiles, and once a frame has been cut into tiles, every 3×3 convolution at a tile border reads invented values. The tile-boundary artefact that follows is what we call the *seam*, and under the default padding rule it costs 0.677 dB on this decoder at high rate. That is several times the entire budget the method works to, and it is paid before any tile has saved a single operation. In Section 4 we treat padding as an *estimator* of the unseen neighbour and measure four of them. The ordering we get is not the obvious one.

A distortion budget only works inside a band.

Tiling costs something even when nothing exits early, so there is a *floor*, and budgets below it admit no allocation at all. The ladder also has a shallowest usable exit, so there is a *saturation* point above which every tile already takes that exit and more quality buys nothing. Between the two, in what we call the *band*, the budget genuinely trades quality for compute. We measure both ends at every rate in Section 5.4, where the 0.1 dB budget uses 45% of the band at low rate and 9% at high rate. A budget, then, is a control variable only inside a window that the method's own geometry creates, and knowing where that window is turns out to matter more than any tuning inside it.

The paper makes three claims and everything else in it is support for one of them: that decoding computation can be allocated by content on a decoder nobody is allowed to change; that a distortion budget is a usable control only inside a window that tiling itself creates; and that the file already says where the computation should go.

Contributions.

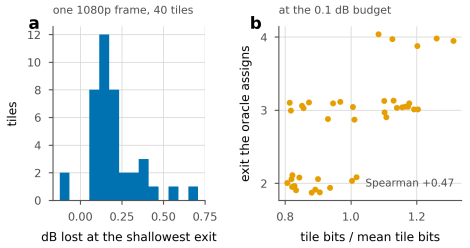

Figure 2. Why depth should not be uniform. One 1080p frame, 40 tiles. **a**, what each tile loses at the shallowest exit the ladder allows: a few lose nothing measurable and one loses 0.71 dB, and a single depth has to be set by the worst of them. **b**, the bits already spent on a tile against the exit the oracle sends it to, at 0.1 dB.

(i) An early-exit ladder for a learned image decoder that picks a depth per tile, which is spatial adaptivity at the granularity of a tile. The encoder, the entropy model and the coded latent are untouched in both modes we report. *Signalled* FLEX-UF leaves the latent unchanged and adds 89 bits per 1080p frame of routing metadata, or 0.021% of a typical bitrate, and saves 29.2% to 15.2% of decoder MACs for 0.1 dB on the common test set at a BD-Rate cost of 1.03%. *Bitstream-identical* FLEX-UF adds nothing at all, decodes a byte-identical file, and reaches 23.6% to 10.7%.

(ii) The band a distortion budget works in: a floor below which no allocation is feasible, a saturation point above which none improves, both in closed form, and seven propositions verified numerically rather than asserted. Measuring the budget in units of that band accounts for most of the rate dependence, on two independently trained checkpoints.

(iii) The bits a tile has already cost predict how deep it has to decode, better than a trained router does. Routing on the entropy model's own output beats our 144 K head at every rate, by up to 3.4 points, with nothing added to the file and nothing learned. It does so while picking the search's exit on fewer tiles than the head does, which says that accuracy over exit labels is the wrong objective and that ordering is what a router has to get right. It is also the zero-learned-parameter control that adaptive-inference work is rarely measured against.

2. Related work

Three lines of work meet in this paper and none of them quite contains it (Table 1). Early exit made depth a per-input decision and left it there; spatially adaptive inference made it a per-region decision but chose between separate networks; and learned compression has spent a decade making decoders cheaper by making them different, which is exactly what a deployed bitstream will not tolerate. What follows sets out what each gives us and what none of them gives.

Early exit.

BranchyNet [20] and MSDNet [3] established the pattern of intermediate classifiers with a confidence rule; SDN [15] framed it as mitigating overthinking, ZTW [54] recycles earlier predictions, and RANet [55] varies input resolution rather than depth. We train all exits jointly, after Scardapane et al. [18], and distil between exits as in Phuong and Lampert [17]. The caveat in [19] is that too large a student-teacher gap hurts the shallowest exits, so our distillation runs between *adjacent* exits. All of this work exits on a confidence signal computed from the network's own output. A decoder has no such signal. There is no class posterior, and the quantity that would decide is the error against a source the decoder cannot see (Section 3.5).

Spatially adaptive inference.

ClassSR [12] sorts super-resolution patches into easy/medium/hard and runs a different network on each; APE [1] exits patches at different depths; Glance-and-Focus [8] spends resolution adaptively. We share the per-region premise with all three, and we inherit the same tiling problem, though the cost of tile borders is rarely quantified in that line of work. To our knowledge the estimator view of border padding and its measured ordering (Section 4) is new. The closest prior work is the per-channel AR(1) padding of Kaseva et al. [11], which we implement and evaluate.

Method	Varies	Where	Latent	Side data	Dec. only
SlimCAE [22]	width	per stream	new	–	no
Slimmable video [23]	width	per stream	new	–	no
EVC [22]	mask / pruning	per model	new	–	no
Spatial competition [27]	which codec	per region	new	yes	no
DCVC-RT [33]	architecture	fixed	new	–	no
FLEX-UF signalled	**decoder depth**	**per region**	**same**	89 b	no
FLEX-UF bitstream-identical	**decoder depth**	**per region**	**same**	**none**	**yes**

Table 1. Where this sits. What each method varies, at what granularity, and what that costs the stream. A single new-bitstream column cannot represent our own two modes, so the last three ask what actually changes: whether the coded latent is the same, what side data travels with it, and whether the decoder alone can do it.

Method	kMAC/px	Par. (M)	BD-Rate (%)
CHARM [51]	496	58.5	+0.86
STF [45]	511	99.9	-2.06
ELIC [44]	574	36.9	-3.22
WeConvene [50]	2343	107.2	-6.98
MambaVC [49]	814	47.9	-8.72
DCVC-DC intra [52]	542	45.5	-9.18
TCM [46]	1824	76.6	-10.70
FLIC [48]	1096	71.0	-13.20
MLIC++ [47]	1283	116.7	-15.15
HPCM-Base [43]	919	68.5	-15.31
HPCM-Large [43]	1261	89.7	-19.19

Table 2. What a learned image decoder costs. Eleven codecs as published, measured on one machine by Li et al. [43]: BD-Rate against VTM-22.0 on Kodak, arithmetic per pixel and parameters. The spread in cost is an order of magnitude, and every entry pays its cost on every pixel of every image.

Method	GMAC	Par. (M)	BD-Rate (%)	Adaptive
DCVC-DC [52]	–	18.3	+14.5	no
DCVC-FM [13]	2642	18.3	-21.3	no
DCVC-RT [33]	385	20.7	-21.0	no
DCVC-UF (LD) [14]	170	9.7	-9.5	no
DCVC-UF (HT-S) [14]	211	81.2	-31.6	no
DCVC-UF (HT-L) [14]	343	120.5	-42.2	no
FLEX-UF intra decoder	**219 → 172**	–	**+1.03**	**yes**

Table 3. The family this work modifies, from DCVC-UF [14]: MACs per 1080p frame, parameters, and BD-Rate against VTM-17.0 low delay. Comparable within this table and not against Table 2, which uses a different anchor and a different test set. The last column is the one this paper is about: whether the decoder spends more computation where the picture needs it. Our row is the intra decoder of the last DCVC-UF entry, at the 0.1 dB budget.

Together the two tables say where the field spends its decoder budget (Table 2, Table 3). Complexity has moved a long way in both directions: TCM [46] and WeConvene [50] buy rate with an order of magnitude more arithmetic per pixel than CHARM [51] or STF [45], and DCVC-UF [14] moves the other way, cutting a 1080p frame from DCVC-FM's 2642 GMAC to 170. Two things are constant down both tables. The cost is a property of the model, chosen once and paid on every image; and it is uniform over the frame, so a flat sky and a face are decoded at the same price. Those are the two the ladder in this paper changes, and it changes them without touching the model that was shipped.

Complexity control in learned compression.

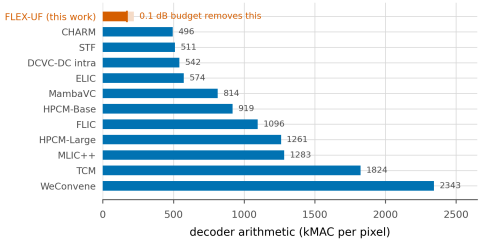

Figure 3. Every published decoder is one number. Arithmetic per pixel for the eleven image codecs of Table 2 and for the decoder this work modifies. Costs are comparable here even where the BD-Rate anchors are not. On the top bar a 0.1 dB budget removes the pale section, by an amount that depends on the picture.

SlimCAE [22] and slimmable video codecs [23] expose several widths of one model. The choice is per stream and it changes the encoder, so the bitstream is not interchangeable. DCVC-FM [13] and DCVC-UF [14] reduce cost architecturally, for every frame equally. Rate–distortion–complexity has since become an explicit third axis. Gao et al. [35] tune spatial context usage to trade decode cost against rate, Ho et al. [36] survey where conditional residual coding sits on that surface, and Zhang and Gao [37] route *whole frames* to one of several jointly-trained coding paths, spending less computation on cheaper frames. Every one of these changes the model, and with it what the encoder emits. Our axis is orthogonal. The model and the bitstream are both fixed, and only *how much of the decoder runs where* varies.

The closest neighbour.

Blard et al. [27] also partition an image into regions, also choose per region by a rate–distortion cost computed at the encoder, and also transmit a mode map. The differences are in what the regions choose among and in what the map buys. Their regions choose among several *separately trained* codecs, so the decoder must hold all of them and its complexity is that of one codec regardless of the choice; the map buys rate, not compute. Ours choose a prefix length of a single trunk, so the weights are shared by construction and the map buys compute at fixed rate. Because their alternatives are unrelated networks, no decoder-side predictor could stand in for the encoder's search. Our exits are prefixes of one another, and that nesting is what lets a decoder-side predictor stand in at all (Section 3.1).

Signalling versus prediction.

Being *told* a mode decision rather than inferring it is the norm in standardised video coding. HEVC [9] and VVC [21] transmit partitioning, prediction mode and transform tree. We evaluate both, and treat the signalled variant as the conventional design (Section 3.1).

Deferring to an oracle under a budget.

Letting a predictor decide most cases and handing a small budget of the hardest ones to something exact is the shape of selective prediction [38] and learning to defer [39, 40], and of the budgeted variant of the latter [41]. We build on that shape in Section 5.7. Two things differ, and both make our case easier. The expert here is the encoder's own search, so it is exact and always available at no test-time cost. What is scarce is the *bits* needed to say what it decided. The selection rule is not learned either. Because the objective is separable over tiles, the optimal set of size s at a fixed multiplier is exactly the s largest regrets, which the encoder can compute. Who to defer and how to learn it is the open question in that literature, and it has a closed form here. What remains is the question we measure, which is how concentrated the regret is (Figure 31).

Allocating a budget over units.

The construction we use is not new and we do not present it as such. Shoham and Gersho [28] showed that for a finite set of per-unit operating points, a Lagrangian sweep decouples the allocation across units and traces exactly the lower convex hull of the achievable set; Ortega and Ramchandran [29] made it standard practice in image and video coding. What we add is the structure this particular operating set has: a floor below which no allocation is feasible, a saturation point above which none improves, and a measurement of what the convex-hull restriction costs (Section 5.4). The same relaxation has resurfaced for test-time compute in language models [30], with per-instance decoupling and a binary search on the multiplier. That is the identical structure in a domain with no rate axis, and we read it as evidence that the floor/saturation characterisation is worth stating generally.

How much is there to gain?

Bounding what adaptive inference could achieve is itself a line of work. Hasan et al. [34] derive an oracle bound on efficiency at fixed accuracy, given per-model resource and accuracy, and report 43–121× on ImageNet and 7–81× on HellaSwag. Their bound has a ceiling and no floor, because the largest model in their family reaches the reference accuracy by definition. Spatial adaptivity introduces a floor. Cutting a frame into tiles costs quality even when every tile runs to full depth, so the reference is unreachable at *any* compute and the feasible set of budgets is an interval rather than a ray. Section 5.4 characterises that interval and both of its ends.

Tile boundaries.

Every method that processes an image in independently-computed tiles meets the same artefact. The remedies in the literature are overlap and averaging, local padding from neighbouring patches [31], training with overlaps [32], and fitted extrapolation [11]. Local padding is the closest to the exact remedy we describe in Section 4.4, and the difference is accounting. It pads every convolutional layer and does not report the cost. We pad only the 0.29% of each block that has any spatial extent, which is what makes the exact fix cost 0.032% of the decode rather than a multiplier on all of it. To our knowledge the estimator view of the padding rule, the measured ordering of four estimators, and the dependence of the penalty on per-tile depth (Section 4) have not been reported.

Where the time goes.

DCVC-RT [33] argues that operational rather than computational complexity is the speed bottleneck for neural codecs, and cites channel

reductions that yield linear rather than quadratic speedups. Section 5.9 is an instance of that claim inside one loop. Removing 35.3% of the operations buys 29.1% of the time, and 1.2 points of the difference come back from reordering the loop with the arithmetic untouched.

3. Method

The construction is short. A frame is cut into tiles once the shared part of the decoder has run (Figure 6), each tile is allowed to leave the remaining trunk at its own depth, and a small adapter reconciles an early departure with a head that was fitted to the full one. Everything difficult is in the three questions that follow from it: what cutting costs, what a budget can buy, and who decides. This section states the construction and the decision rule; Section 4 prices the cut and Section 5 measures the rest.

3.1 Setting and notation

We set out here the decoder we work on, the notation the rest of the paper uses, and the three configurations we compare.

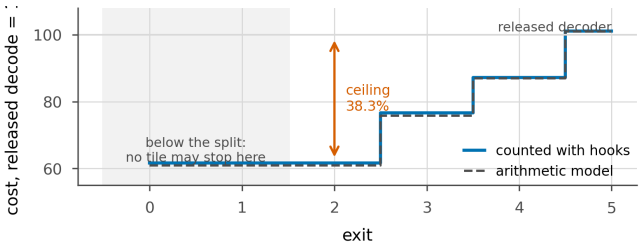

Figure 4. The ladder, priced. What a tile costs at each exit, as a percentage of one released decode, counted with hooks on the executed pass and beside the arithmetic model the argmin uses. Exits below the split depth are not distinct, because the first j groups run for every tile whatever it does. The deepest exit costs slightly more than the release, which is the full-frame deblocking pass. Reported savings in this paper are the hook count.

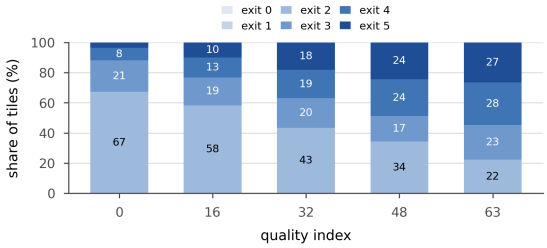

Figure 5. Where the tiles actually go, at the 0.1 dB budget, pooled over the whole test set. At the lowest rate two thirds of tiles take the cheapest available exit; at the highest only a fifth do and a quarter need the deepest. The allocation is not degenerate at either end, which is what makes the choice worth making.

The decoder.

We work on the intra decoder of DCVC-UF [14]. The analysis side stays frozen throughout: the patch embedding, the chunk encoder, the entropy model and the coded payload are never touched (Figure 1). What we replace is the synthesis trunk that turns the decoded latent into a picture. We call the unmodified public decoder the *released decoder*, and one full-frame run of it is the unit of both cost and quality below.

Tiles and exits.

A frame is cut into square tiles, 256 px on a side unless we say otherwise, and each tile is carried through part of the trunk on its own. Tiles are indexed by t and there are N of them, N=40 at 1080p. The trunk itself is a stack of twelve blocks; we cut it into K groups and attach an exit to each, so exits are indexed by k and exit K−1 is the whole trunk. The first j groups run once over the whole frame and the remaining K−j run per tile, so j is the *split depth*. A tile that leaves at exit k runs groups j to k and skips everything after it. Writing c_k for what a tile costs at exit k, a frame that assigns exit k_t to tile t costs the mean of c over its tiles. Tiles are cut on the trunk's feature grid, which is coarser than the pixel grid, but we quote tile sizes in pixels.

Distortion and the budget.

D(t,k) is the error tile t incurs if it leaves at exit k. It is measured on the path we deploy at inference, a tiled decode, against the released decoder's full-frame decode of the same latent, so what tiling costs is inside it; we call that path the *deployed path* and the table built on it the *deployed table*. Tapping the exits of one full-frame forward pass gives a cheaper *full-frame table* that is not the same quantity, and Section 5 measures the difference. Every result is quoted at a *distortion budget*, the decibels a decoded frame is allowed to fall below the released decoder on that same latent, and the headline budget is 0.1 dB. To allocate is to choose an exit for every tile so that the frame lands on its budget as cheaply as possible. A budget only buys something inside its band, because tiling costs quality before any tile exits early and the shallowest usable exit bounds what can be saved; Section 5.4 measures both ends. Rate is set by a quality index running from q0, the lowest rate, to q63, the highest, and we report five of them: q0, q16, q32, q48 and q63.

symbol	meaning
t, N	tile index and tiles per frame; N=40 at 1080p
k, K	exit index and number of exits, k in 0..K−1; K=6
j	split depth: groups 0..j−1 run full frame, j..K−1 per tile; j=2
c_k	cost of a tile at exit k, as a fraction of one released full-frame decode
D(t,k)	error of tile t at exit k, tiled decode against the released full-frame decode of the same latent
λ	multiplier trading c_k against D(t,k), bisected to the budget
β	the same trade applied to the router's logits
q	quality index, q0 the lowest rate and q63 the highest

Three configurations.

All three choose the same object, one exit per tile, on the same weights and the same coded payload. Configurations **A** and **B** differ only in where that choice is made, and **C** sits between them.

A decides at the encoder. The encoder holds the source, so it knows D(t,k), picks each tile's exit by the Lagrangian of Section 3.5, and transmits the resulting *exit map*, one exit index per tile (Figure 5), which is the analogue of the mode map a standard codec signals. We report its cost as the entropy of the map's own per-frame histogram, 89 bits for a 1080p frame, which is 0.021% of a typical bitrate; that is what an arithmetic coder would spend on it rather than the two-bits-per-tile worst case, and it is an upper bound in one respect we have left in place. Below the split depth the exit indices are not distinct, so a real codec would signal four symbols where this entropy counts six. Measured, that inflates the figure by at most seven bits a frame. The map costs the encoder about one extra decode per frame and the decoder nothing.

B decides at the decoder. A 144 K-parameter head reads decoded data only and predicts the same map (Section 3.5). Nothing is transmitted and the file stays byte-identical to the released one. It costs the decoder 0.163% of its own multiply-accumulates, which is charged inside every B number we report. Because that head reads nothing the encoder cannot also read, the encoder can run it too, and so knows tile by tile where it will be wrong.

Two of these are deployment modes and we name them so that no claim in this paper is ambiguous about what is sent. *Signalled* FLEX-UF is configuration A: the coded latent is unchanged and a small exit map travels beside it. *Bitstream-identical* FLEX-UF is configuration B: nothing is added, and the decoder reads a file that is byte for byte the one the released encoder produced. Every saving quoted in this paper is labelled with the mode it was measured in.

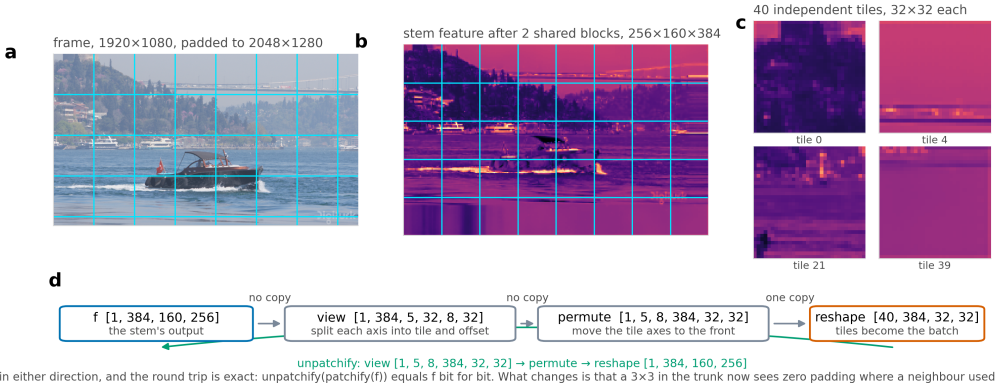

Figure 6. What patchify does. From one real decode. **a**, the frame padded to whole tiles, with the grid the decoder will impose. **b**, the feature map after the shared stem at one eighth resolution, so a 256 px tile is 32×32 of features: the tiling happens here, not in the pixel domain. **c**, four of the 40 tiles as the trunk sees them. **d**, the operation and its inverse, which costs no arithmetic and is exact.

C interpolates between the two. The encoder signals a fraction ρ of the tiles, the ones where leaving B alone is most costly, and B decides the rest, so ρ=0 is B and ρ=1 is A. Any decoder-side predictor can take B's place there, and Section 5.7, which measures C, tries a second one. C transmits a mask rather than a full map, costs the encoder A's search plus one run of the predictor, and costs the decoder whatever that predictor costs.

3.2 Where the computation is

	q0	q32	q63			
Exit	with	without	with	without	with	without
2	0.177	2.065	0.228	2.884	0.297	4.398
3	0.089	1.023	0.123	1.772	0.151	3.054
4	0.056	0.416	0.077	0.574	0.084	0.853
5 †	0.036	0.036	0.048	0.048	0.056	0.056

Table 4. What the adapters are worth. dB below the released decoder with every tile at that exit, with the trained adapters and with each set back to the identity it was initialised to. † the deepest exit has no adapter by construction and is the control.

The DCVC-UF intra decoder is one upsampling block, twelve DepthConvBlocks and a head, costing 453.5 GMAC per 1080p frame. The twelve blocks are \89.4% of that, which is why we build the ladder across them. Inside one block at C=384 channels, the only operator with any spatial extent is a 3×3 depthwise convolution, and it costs 9C against the block's 8C²+9C. That is **0.29%.** We come back to the fraction several times below. Tile borders damage the picture through it, and it is what makes the exact remedy of Section 4.4 affordable at all. It is also why we kept the adapters pointwise.

3.3 The exit ladder

Why the tile is 256 pixels. Two constraints bracket the choice and neither is free to ignore. Below, the tile has to be large enough that its border is a small part of it. The seam grows with the fraction of a tile within reach of an invented value, 1-((F-2b)/F)² for b per-tile blocks on a tile of side F, so halving the side roughly doubles the damage at fixed depth (Section 4). Above, it has to be small enough that a frame holds enough of them for an allocation to exist at all: at 256 px a 1080p frame is 40 tiles and a 416×240 frame is 2, and the second is already close to having no choice to make (Section 5.3). Standardised codecs bracket their own partition the same way and land nearby, at 64 pixels in HEVC [9] and 128 in VVC [21], for reasons that include this one. We measured 128 against 256 and report it, with the caveat that the two come from different training runs.

Two consequences follow from the ladder, and both are easy to state wrongly. Exits shallower than j are indistinguishable from each other, because the first j groups run for every tile regardless, and the usable ladder therefore has K−j distinct costs (Figure 4). The second consequence is the *architectural ceiling* S_max = 100(1−c_j)%, reached when every tile takes exit j. For our shipped setting (K=6, j=2, 256 px tiles) S_max = 38.3%. We show in Section 5.4 that this bound is *reached* in practice at low rate, so it behaves as an operating point and not as an asymptote.

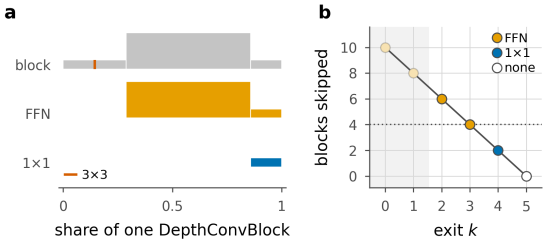

Figure 7. Inside an exit adapter. a, one DepthConvBlock and the two adapters, drawn to scale: bar length is a share of the block, bar height the channel width written. Neither adapter contains the block's only 3×3 (the hairline rule), so neither adds receptive field or seam penalty. **b**, blocks skipped per exit, coloured by adapter; the rule switches to the FFN at four skipped blocks. Lengths counted with hooks off the modules themselves.

3.4 Exit adapters

An early exit hands the shared head a feature the head was not fitted to, and the adapter is the correction. For exits that skip little we use a residual 1×1, Ad(f) = f + Wf with W zero-initialised. For exits that skip four blocks or more we use the pointwise expand/activate/contract pair of the block's own FFN (Figure 7). The threshold is where the capacity mismatch stops being ignorable: a 1×1 costs C² per pixel and each block it stands in for costs 8C²+9C, so at two skipped blocks it is replacing sixteen times its own arithmetic and at six it is replacing fifty. Both are *pointwise by design.* A 3×3 inside an adapter would add seam damage at the tiles that took an early exit, and those are the tiles least able to afford it. Zero-initialising the last layer makes every adapter exactly the identity at step zero, so the deepest exit is bit-exactly the released decoder before training begins and every shallow exit starts from “the decoder as it is”.

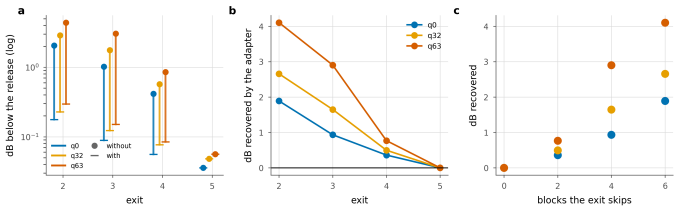

Figure 8. What the adapters are worth. a, each exit with and without its adapter. **b**, the dB the adapter recovers. **c**, the same against the number of blocks the exit skips, which is the design rationale measured.

Zeroing the adapters measures what they learned, since the identity is exactly what they were initialised to (Figure 8) and Table 4. Without

them the shallowest exit costs 4.40 dB at q63, forty-four times the budget, and the adapter buys 4.10 dB of that back. We find the gain grows with rate and with the number of blocks the exit skips, so the design rationale here is a measurement rather than an argument, and the deepest exit moves by exactly zero. We would not describe the adapters as a refinement of the ladder, because without them it has no usable exit.

3.5 Allocation

The ladder and its costs are fixed by this point (Figure 9), and one decision is left. Every tile has to be given an exit, drawn from the usable ones, $k \geq j$, and we want the set of choices that keeps the frame inside its distortion budget for the least compute. There are K–j choices per tile and N tiles, so searching assignments one by one is out of the question. What makes the problem tractable is that the tiles do not interact: each contributes its own error D(t,k) and its own cost c_k, and the only thing they share is the single budget the frame has to meet.

Problems of that shape are solved with a Lagrangian. Put a price λ on compute, add λ c_k to the error of exit k, and the budget constraint drops out; what is left is N separate one-tile problems, each a minimum over K–j numbers. Shoham and Gersho [28] showed that sweeping the price this way traces the lower convex hull of what the units can achieve together, and Ortega and Ramchandran [29] made the construction standard in image and video coding.

What λ does is set the exchange rate between compute and error. At λ=0 compute is free and every tile takes whichever exit has the smallest error. As λ rises, compute becomes expensive relative to error, and a tile drops to a shallower exit as soon as what that exit saves is worth more than the error it adds. Compute falls and error grows as λ grows, neither of them turning back, so one value of λ puts the frame exactly on its budget and we find that value by bisection. The rule each tile follows is

$$k^*(t) = \arg\min_{k \geq j} \left[D(t,k) + \lambda c_k \right] \tag{1}$$

with the minimum taken over $k \geq j$. We call k*(t) the choice of the *Lagrangian oracle*: it is made with the true error of every exit in hand, which no decoder has, but it is optimal only within the family of allocations one multiplier can reach. That qualifier is not cosmetic. Section 5.7 exhibits allocations that are cheaper at the same distortion than anything this rule produces, so *oracle* here means best under a single multiplier and not the global constrained optimum. Where we mean the latter we say so. Both quantities are available *to the encoder*, which holds the source, so configuration A can run this search exactly.

What the search costs the *encoder* depends on how the table of D(t,k) is built, and the two ways of building it are further apart than they look. Take one tiled decode at 1080p as the unit. The deployed table of Section 3.1 needs one tiled decode per exit, which comes to 4.6× the unit rather than K times it, because a shallow exit is cheaper to run than a full one. The full-frame table taps every exit off a single full-frame pass and comes to 1.4×. Section 5 shows that the full-frame table must not be used to *report* quality, but it can still be used to *rank*. Running the argmin on it, and the deployed path only to check the budget, we find the two searches agree on 85% of tiles and land within 0.4 points of saving of each other (19.0% against 18.6%, both under budget). The exact search therefore costs 4.6 decodes and a practical encoder need not pay it; ranking on the full-frame table is the one extra decode per frame we quote for A.

The decoder cannot run the argmin. D(t,k) is the error against the source, and the source is the one thing a decoder never receives. The gap is one of information: the errors the argmin compares depend on a picture that is not in the bitstream, so no decoder-side model recovers them, however large it is. What a decoder can do is estimate which exit the Lagrangian oracle would have picked, and configuration B therefore *predicts*. Its head reads what the decoder already holds, the feature at the split point, the decoded latent y-hat, the entropy model's scales and the quality index. For each tile t it emits a vector of logits z_t, one entry per exit, and the tile takes

$$\hat{k}_t = \arg\max_{k \geq j} \left[\log \operatorname{softmax}(z_t)_k - \beta c_k \right] \tag{2}$$

The first term is the head's score for exit k, on a log scale. The second is the price on compute that λ carries in the Lagrangian: raising β takes more away from the deep exits than from the shallow ones, so more tiles are handed to cheap exits and the frame again gets cheaper and worse. We bisect β against the budget just as we bisect λ.

Where β comes from at deployment. A real decoder cannot run that bisection. It would need to know the delivered distortion, and the distortion is measured against a source the decoder never receives, which is the same missing variable that stopped it running the Lagrangian in the first place. Saying otherwise would smuggle the source back in at test time.

So β is not chosen at test time. It is calibrated offline, once, on a held-out set, and shipped as a table indexed by the quality index and the budget. The decoder reads the quality index out of the bitstream, looks β up, and runs the argmax above; nothing in the loop needs the source, and nothing needs a relaxation of the discrete choice [10] either, because the head is supervised on the search's label rather than trained through the decision. The table is 64 quality indices by however many budgets a deployment offers, which is a few hundred floats.

We calibrate it on the 512 held-out Open Images validation frames, which are disjoint from the training images and from the test sequences: β is bisected to the budget on those, once per quality index, and the resulting table is then applied unchanged to the test set. Section 5.5 reports both that number and the one obtained by bisecting on the test set itself, because the difference between them is the only honest measure of whether the table transfers.

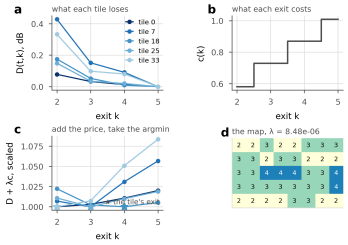

Figure 9. How a multiplier becomes a map. Five tiles of one real frame. **a**, D(t,k), what each tile loses at exit k in dB against the deepest. **b**, c(k), what each exit costs as a fraction of one released decode. **c**, the two added with the multiplier that meets a 0.1 dB budget, each curve scaled to its own minimum; the star is where the argmin lands and that is the tile's exit. **d**, the map the same rule produces for all 40 tiles. One number, λ, decides the whole frame.

Why a logarithm. The oracle's rule adds a distortion to a price. The head does not produce a distortion; it produces a distribution over exits, and the quantity that plays the same additive role is the surprisal –log p, which is what the head is trained to make small on the oracle's label. Writing the rule with log softmax puts the two terms in one currency: what exit k costs in belief against what it costs in compute, with β the exchange rate between them. It also makes the comparison invariant to a constant added to every logit, which the softmax removes, so βc_k is the only thing tilting it. Using the probability itself would set a number bounded in [0,1] against a cost measured in decodes, and would compress every distinction between confident exits into a range near one, where β would have to move by orders of magnitude to change any decision.

The bracket for the bisection is $[-2\times10^4, 2\times10^4]$, which is wider than it looks like it needs to be. A trained head is confident: its log-probability gaps reach the hundreds, and β has to be able to outweigh them before the allocation moves at all. A narrower bracket we tried first, [–50, 50], never reached the all-deepest allocation and so never bracketed the budget from below.

The head, exactly. Figure 10 draws it. Three things enter. The first is the feature at the split point, 384 channels at one eighth of frame resolution, which is the last thing every tile shares. The second is the decoded latent ■ concatenated with the entropy model's scales σ [2], the predicted Gaussian width used to code each latent position: σ is already computed during the decode and is, position by position, an estimate of how hard that position was to code. The third is the quality index. Each of the first two is projected by a 1×1 convolution, to 48 and 32 channels, and then reduced to one vector per tile by taking the mean and the standard deviation over the tile. Both moments are there on purpose: the channel means describe roughly a tile's colour and the standard deviations its texture, and texture is what decides how many blocks a tile needs. The same pair is what adaptive instance normalisation takes as a compact description of a feature map's style [53]. That gives 96 + 64 + 1 = 161 numbers per tile, which pass through LayerNorm and a three-layer perceptron of width 256 with SiLU activations, ending in K logits. The whole head is 144,030 parameters and \0.163% of the decode it is deciding about, almost all of it in the 1×1 on the stem, which is the only part that runs per pixel. The perceptron runs once per tile, forty times for a 1080p frame, so its width is nearly free.

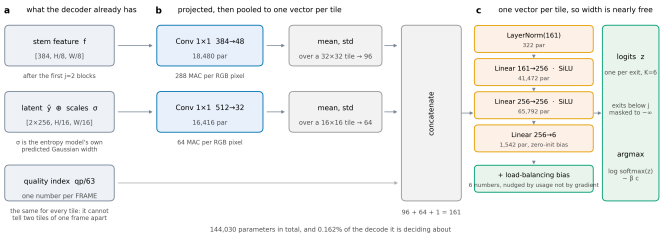

Figure 10. The router head. a, the three things it reads, all already held by the decoder. **b**, each projected by a 1×1, then reduced to one vector per tile by its mean and standard deviation. **c**, a three-layer perceptron, one pass per tile. Every shape and parameter count is read off the module when the figure is drawn.

Two details matter for reproducing it. Exits below the split depth do not exist, and their logits are masked to $-\infty$ rather than to a large negative constant; nothing in the objective penalises a constant offset, the raw logits drifted to around -10^4 during training, and a -10^4 mask then became the *largest* entry in the row. The balancing term is a per-exit bias added to the logits before the decision and updated by usage rather than by a gradient, which is what makes it loss-free.

One head for every quality index. The index is an input, not a selector. During training it is drawn uniformly at random for each batch, so one set of weights covers all 64 operating points, and at test time the same head runs at every rate. It enters as a single number per frame, which means it can tell the head which operating point it is at and cannot, even in principle, distinguish two tiles of the same frame. That is why it stays live in every variant of the input ablation: the variants then differ in per-tile information and in nothing else, and the variant with only the quality index is the control that measures what agreement is reachable with no per-tile information at all.

The head is trained against a *frozen* decoder, so the exits it chooses between do not move while it learns. Its target is the oracle's choice k*(t) and the loss is a cross-entropy to that target, with each tile weighted by its *regret*, meaning by how much the bracket in the Lagrangian grows if the head's exit is used in place of the oracle's. A tile whose two best exits are nearly tied then counts for less than one where the wrong choice is expensive. A router like this can collapse onto a single exit during training, so it carries a balancing term; ours is loss-free [16], unlike the auxiliary loss a mixture-of-experts router carries [7], and is biased toward the oracle's own exit distribution instead of toward uniform. At a high λ the oracle genuinely does send every tile to one exit, and forcing spread there would force mistakes. Section 5.5 measures what B gives up against A.

3.6 Training

All exits are decoded every step and the objective is

$$\mathcal{L} = \mathcal{L}_{\text{RD}} + w_a\,\mathcal{L}_{\text{anchor}} + w_d\,\mathcal{L}_{\text{distill}} \tag{3}$$

where L_RD is the released rate-distortion loss averaged over exits, weighted per exit by fixed α_k and scaled by λ_rd, the codec's own rate-distortion weight, which is a different multiplier from the λ of the Lagrangian above. The anchor term L_anchor pins the deepest exit's reconstruction to the released decoder's, so the reference every saving is quoted against cannot drift away underneath the measurement. The distillation term L_distill supervises the adapters in feature space, matching the feature at exit k to a stop-gradiented copy of the feature at exit k+1. We work in feature space and not in pixels because the head is a fixed map from feature to RGB, so matching the deeper feature is the stronger constraint, with 384 dense channels of target instead of 3. Each exit imitates its neighbour, exit k following exit k+1, and not the deepest exit, for the reason given in [19].

We train the adapters *through the tiled decode path they are deployed in*. Training them full frame and tiling only at inference loses 0.14–0.24 dB; training through the deployed path gains 0.51–0.90 dB, so the sign of the effect flips.

4. The price of tiles

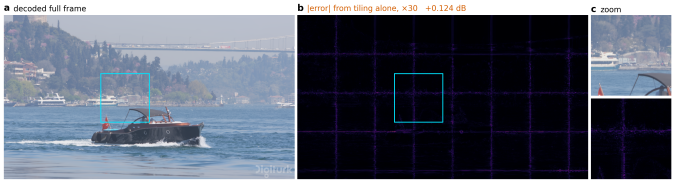

Figure 11. The artefact, before anything is done about it. Bosphorus at 1080p, quality index 63, zero padding at the tile border. **a**, the frame decoded in one piece. **b**, the same bitstream and weights decoded in 256 pixel tiles, every tile at full depth: the difference between the two, amplified 30 times. **c**, the boxed region of each.

Cutting a frame into tiles is what makes per-tile depth possible and it is not free, so before any saving is claimed the cut has to be priced. This section does that, and the price is larger and stranger than the literature on tiled inference suggests.

A 3×3 depthwise at feature position (x,y) computes a weighted sum over its neighbours. Decoded full frame, those neighbours exist. Decoded per tile they do not, and the kernel is handed whatever the padding rule invents. Each convolution extends the affected region by one ring, so with b per-tile blocks on a tile of side F the fraction of the tile within reach of an invented value is

$$1 - \left(\frac{F-2b}{F}\right)^2$$

At our shipped F=32, b=8 that comes to 0.750. Three quarters of the tile is affected, so what we are looking at is a structured error over most of the tile (Figure 11) and not a thin border at its edge.

That fraction tells us which pixels are affected and not how badly, and it turns out to be the wrong predictor of the penalty. We swept the split depth, which sweeps b from 12 to 0 with b=0 as an exact zero-seam control, and measured

$$\Delta_{\text{seam}} \propto b^{\alpha}, \qquad \alpha \approx 2 \tag{5}$$

with α = 2.38 at q0, 2.22 at q32 and 1.93 at q63, and r = 0.98–0.995 in log-log. Fitted with one free scale, the area fraction is wrong by 160–264% where the power law is wrong by 12–22%. Fixing the exponent at exactly two costs a factor of two, 31–38%, so the square is the right shape without being quite the right law. The exponent has a reading. The count of affected pixels grows like perimeter times depth, and the error accumulated in each of them grows with how many convolutions reached it. Once every pixel has been touched the area law

saturates and the penalty does not, which is where the area law fails as a predictor.

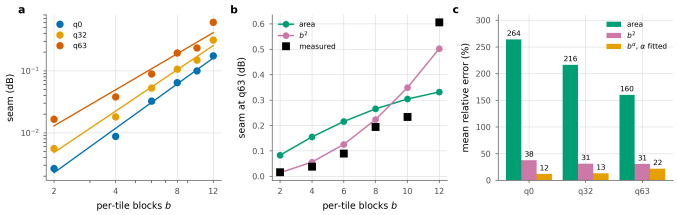

Figure 12. The seam against per-tile depth. a, the measurement with the fitted power law. **b**, both models against q63, one free scale each. **c**, mean relative error. The area fraction saturates once the border reaches every pixel; the penalty does not.

4.1 Padding is an estimator

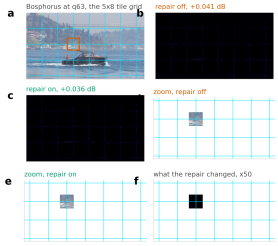

Figure 13. What the deblocking pass does to a picture. Bosphorus at q63, the rate where the seam is worst, decoded twice from one latent at full depth. **a**, the frame and its 5×8 grid. **b**, **c**, error against a full-frame decode, ×25, pass off and on. **d**–**f**, one grid crossing and what the pass changed there. It recovers 0.005 dB of the 0.041 dB tiling costs on this frame.

Border padding is an *estimator* of the unseen neighbour, and the seam is its error (Figure 12). The table below measures four of them, with early exit switched off (Table 5) so that tiling is the only difference from a full-frame decode.

Border estimator	q0	q32	q63
Zeros (stock)	0.126	0.287	0.677
Replicate	**0.071**	**0.123**	**0.218**
Linear extrap.	0.198	0.286	0.384
AR(1) per channel [11]	0.061	0.107	0.197

Table 5. Border estimators, dB below the released decoder on the same latent, 256 px tiles, full CTC. Lower is better. Replication, a zero-order hold, beats linear extrapolation by a wide margin, and the fitted AR(1) wins on quality while costing +10.7% of decode wall-clock.

A higher-order estimator is worse. Extrapolating the local gradient past a boundary amplifies whatever noise sits on that boundary, and assuming local constancy does not. The gap is wide, 0.384 dB against 0.218 dB at q63. **The best estimator loses on cost.** The per-channel AR(1) fit of [11] reaches 0.197 dB, about 0.02 dB better than replication, and it costs 10.7% of decode wall-clock. Against a 0.1 dB budget and a ~24% saving that trade does not close, so we drop it.

4.2 What actually removes the seam

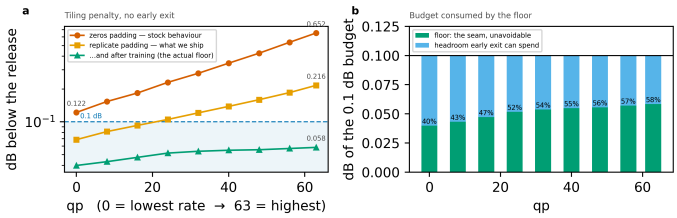

Figure 14. The tiling penalty across the whole rate range, every tile at full depth; the left axis is logarithmic. The two steps that matter cost nothing, and the largest single factor is training the ladder with the seam present. Panel (b): the floor is charged *inside* the distortion budget and consumes a growing share of it.

If padding is the estimator, the obvious next question is what a better one is worth, and the answer is: less than changing the geometry. Tile size is free in arithmetic. A tiled decode's MAC count does not depend on it at all, and the damage scales with the tile perimeter (Table 6), as the table below shows. What tile size costs is routing granularity, 40 tiles per 1080p frame at 256 px against 160 at 128 px.

Estimator, q63	128 px	256 px	ratio
zeros	1.249	0.677	×0.54
replicate	0.372	0.218	×0.58
arls	0.332	0.197	×0.59

Table 6. Tile size, dB at q63. Doubling the side roughly halves the penalty, as the power law above predicts, and costs no computation.

The largest factor of all is the easiest one to miss. On the untrained ladder, replicate padding leaves 0.218 dB at q63; after training, the same setting leaves 0.072 dB. So more than two thirds of what the estimator could not fix is absorbed by weights learning to live with it (Figure 14). That single change removes more of the seam than any module in this paper does.

4.3 A learned deblocking filter, and why it is not sufficient

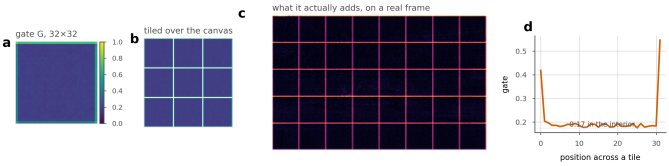

Figure 15. The deblocking gate, and what it does. a, the trained gate G, one scalar per position within a tile, shared across all 384 channels. **b**, the same gate tiled over the canvas, which is how it is applied. **c**, the correction it adds on a real frame: the tile lattice and nothing else. **d**, a cut through a tile. The gate reaches 0.76 at a corner and flattens at 0.17 rather than switching off.

What a standard codec does about a partition boundary is deblock it, with a filter applied after reconstruction [9, 21] (Figure 15). The tile lattice is known exactly at training and at inference, so ours can be *told* where to look instead of having to infer it

$$\mathrm{Rep}(f) = f + G[x \bmod P,\ y \bmod P] \cdot \mathrm{PW}(\mathrm{WSiLU}(\mathrm{DW}_{3\times3}(f))) \qquad (6)$$

PW, DW	a pointwise convolution and a depthwise one
WSiLU	the weighted SiLU of the DCVC line [33]
G	a P×P gate, shared over all channels
P	the tile pitch, in feature samples
G at step 0	exp(−d/τ), with d the distance to the nearest tile boundary and τ a fixed decay length

The whole pass costs 0.95% of the decode.

It does not earn that. Splitting the per-pixel error by distance from the nearest tile boundary, we find the filter gains 0.27% in the 0–4 px ring, which is 6.2% of pixels, and loses 0.04–0.05% everywhere else. Give it a *perfect* gate, with zero correction in the interior and the boundary gain unchanged, and the ceiling on what it could earn is ≈0.0008 dB, for 0.95% of the decode. We rejected AR(1) padding at 0.0019 dB per point of decode, and this is worse by an order of magnitude. Tightening the gate cannot rescue it, because there is almost nothing left to win.

4.4 Removing the cause, and why it does not help

Only 0.29% of each block has spatial extent, so the exact fix is affordable (Figure 13). Give the 3×3 its real neighbours across the tile border, a *halo exchange* narrowed to the one operator that needs it; local padding [31] does the same at every layer. The cost is +0.032% of the decode, thirty times less than the deblocking filter. The fix is also exact. At uniform depth a tiled decode with the exchange is bit-identical to a full-frame one, once the comparison is not confounded by the filter, which runs on the stitched frame and has no full-frame counterpart.

Measured: 1.13e-2 with the filter on, **exactly 0** with it off, against 6.06e-2 for replicate padding.

The exchange also removes most of the floor. Switched on at inference, it drops the floor from 0.036 to 0.003 dB at q0 and from 0.056 to 0.030 at q63, which is 91% and 47% of the tiling penalty.

And it destroys the allocation. At the same 0.1 dB budget the saving falls from 25.9% to 4.2% at q32 and from 19.2% to 0.4% at q63.

The reason is structural, and it is written into the condition on the exactness claim. The exchange is exact when every tile is at the *same* depth, and routing deliberately violates that condition. With replicate padding a tile is entirely independent of its neighbours, so the allocation is free to give adjacent tiles any depths it likes. The exchange makes a tile depend on its neighbours' features, and under routing those neighbours ran a different number of blocks. A shallow tile reading a deep neighbour's activation is an arrangement the weights have never seen, and that mismatch costs more than the seam it removed.

The natural reading of the exactness result is that the exact fix should simply be adopted, and that reading is wrong. We report the negative result for that reason, and because the tension between the halo exchange and routing belongs to the combination itself, so any spatially-adaptive decoder will inherit it. One caveat keeps the finding from being final, the same one that applies to the deblocking filter. This model was trained with padding, and a run trained *with* the exchange could reverse the result. The burden of proof lies with that run.

5. Experiments

The measurements answer the questions the method leaves open, in the order it leaves them. What does adaptivity buy over the alternatives that need no ladder at all; where does a budget stop working; what does it cost to decide on the decoder's side instead of the encoder's; and what does any of it come to in seconds and joules rather than arithmetic.

Setup.

We fine-tune the decoder on 512×512 crops from OpenImages with the encoder frozen (max|Δ| = 0 is asserted every run). One λ_rd is drawn per sample from a log-spaced range covering all 64 quality indices, so a single set of weights covers the whole rate range. We evaluate on one intra frame from each of the 53 sequences of the common test set (CTC: UVG, MCL-JCV and HEVC classes B, C, D and E), one intra frame each.

Measurement protocol.

Decoder	GMAC	% of release	ms
Released DCVC-UF	454	100.0	111
FLEX-UF, all tiles deepest	458	101.0	114
FLEX-UF, 0.1 dB budget	356	78.5	78
FLEX-UF, architectural ceiling	263	58.1	–

Table 7. Decoder complexity at 1080p. Our deepest exit costs slightly more than the released decoder because it still pays the deblocking filter; that 1.0095, not 1.0, is what every saving in this paper is *not* divided by. Wall-clock is the sorted loop of Section 5.9.

Two details of the protocol move the numbers enough that we state them here. The per-tile distortion table has to be built on the *deployed* decode path, that is, one tiled decode per exit, rather than by tapping the exits of a single full-frame forward pass. Tapping is the natural implementation and it is correct for training. But with a full-frame reference on the other side of the ratio, the tiling penalty cancels, and the quality reported is that of a decoder nobody ships. On our model the gap is +0.035 dB at the deepest exit and +0.008 dB at the shallowest. It is not a constant offset; it grows with how many blocks ran per tile, so we cannot correct for it after the fact. The second detail is that once the allocation is chosen, we decode that *mixed* map once and report the distortion it actually produces.

Reporting conventions.

Every dB is measured against the *released* decoder's full-frame decode of the *same* latent, and our side is the deployed tiled decode, so the tiling penalty sits inside every number. Savings are fractions of the released decoder's cost. Our own deepest exit costs 1.0095 of it, and using that as the denominator would flatter every result by 0.6–0.8 points. Rate-distortion differences are integrated as BD-Rate [4] over the five quality indices. Means are taken over unrounded values, so averaging a printed row can differ from the printed mean by a unit in the last place.

The protocol every number in this paper obeys. Two implementations of the same quantity have differed here by as much as 3.29 saving points, and the choice between per-frame and pooled distortion moves an integrated saving by 7 to 10, so the convention is not a detail and we fix it once.

reference	the released DCVC-UF decoder, run full-frame on the same latent
distortion	MSE pooled over the frame, then converted to dB against that reference, then averaged over frames
budget	applied per frame, not to the set mean
λ and β	one scalar per (quality index, budget), not per frame and not per tile
the map	re-decoded: every reported dB is a real decode of the mixed-depth map, never the uniform-exit table
saving	hook-counted MACs of that decode over the released decode's, on the same latent

Table N. The measurement protocol. Every number in this paper is measured this way. The supplement varies each line in turn and reports what the alternative costs.

5.1 Main result

Budget	q0	q16	q32	q48	q63	mean
0.10 dB	29.2	25.0	20.4	17.9	15.2	**21.5**
0.20 dB	38.3	37.0	33.0	31.0	28.9	**33.7**
0.30 dB	38.3	38.3	38.3	37.1	35.0	**37.4**
0.50 dB	38.3	38.3	38.3	38.3	38.3	**38.3**

Table 8. Decoder MACs saved (%) against the released decoder, per quality index and distortion budget, configuration A. The 0.3 and 0.5 dB rows sit on the architectural ceiling almost everywhere; past that point a looser budget buys nothing.

A tenth of a decibel is a convention, not a threshold. We chose it because it is small against the spacing of the rate points and because codec work has long used differences of this size as a working tolerance. It is not evidence that the difference is invisible, and we make no perceptual claim for it. The 0.2, 0.3 and 0.5 dB rows are there so a reader who disagrees with the choice can read off another one (Table 8). We say this before the numbers because every number in this section is conditioned on it.

The table carries the headline numbers. At 0.1 dB the method saves 29.2% at the lowest rate and 15.2% at the highest, and 21.5% on average, at a BD-Rate cost of 1.03%; that is, the compute is worth about the same as a 1.03% increase in bitrate. We do not read the fall with rate as an artefact of the ladder. High-rate reconstructions carry detail the shallow exits cannot reproduce, and the floor rises with rate as well; both effects push the same way.

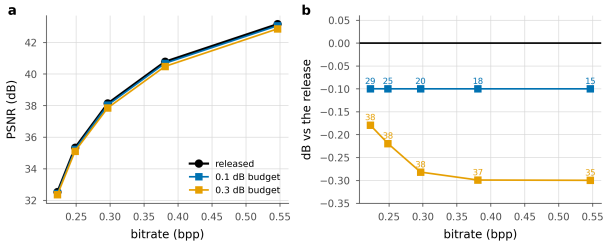

Figure 16. The plane a codec is read on, and the spread behind the mean. a, operating points against the released curve. **b**, the same with the quality axis expanded; labels are compute saved. **c**, per sequence at a matched point near 0.1 dB; one dot per sequence, bar is the median.

Panel c shows the distribution that the headline averages over, and it is wide (Figure 16). At q0 the median sequence saves (Figure 20) 35.9%, with an interquartile range of 28.8–38.8, while the worst saves -1.0%. The worst cases are the low-resolution sequences of Section 5.3, where two tiles leave nothing to allocate. Reporting the mean alone would hide both ends.

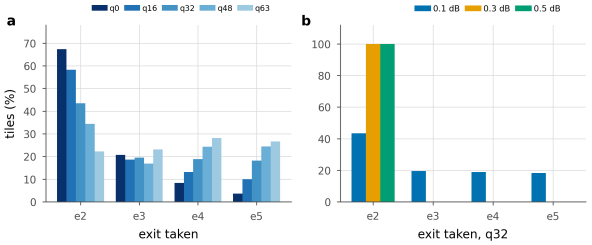

Figure 17. Where the ladder is used. Tiles per exit over the whole test set. **a**, the 0.1 dB budget at five rates. **b**, one rate at the three budgets. Exits below the split depth do not exist and are not drawn.

The fall in saving with rate is the allocation walking down the ladder (Figure 17). At the lowest rate 67% of tiles take the shallowest exit that exists and 4% take the deepest; at the highest those are 22% and 27%, and the mean exit moves from 2.48 to 3.59. A high-rate reconstruction carries detail the shallow exits cannot reproduce, so the same decibel buys fewer rungs. Panel b is the saturation of Section 5.4 in the same units: at 0.3 dB and above every tile is already at the shallowest rung the ladder offers, and a looser budget has nothing left to buy. That is why the 0.3 and 0.5 dB rows of Table 3 differ by less than a point.

What a set-level budget hides

One multiplier is bisected for the whole test set, so the budget binds on the set mean and on nothing else. Sequence by sequence it does not bind at all. Between 29 and 34 of the 53 sequences receive a decode worse than the 0.1 dB they were nominally given (Table 9), and the worst single sequence is 0.354 dB, three and a half times the budget. Nothing is wrong with the bisection; this is what a mean is. We report it because a deployment that needs a per-sequence or per-frame guarantee has to bisect per sequence or per frame, which the encoder-side configuration can do at the cost of one search each, and because a saving quoted against a set-level budget is not the same quantity as one quoted against a guarantee.

q	over budget	worst dB	on	least saved %	on
q0	34 of 53	0.245	videoSRC02	6.0	BQSquare
q16	33 of 53	0.311	videoSRC02	-1.0	BQSquare
q32	32 of 53	0.354	videoSRC29	-1.0	PartyScene
q48	29 of 53	0.290	videoSRC21	-1.0	BlowingBubbles
q63	33 of 53	0.237	videoSRC02	-1.0	RaceHorses

Table 9. The 0.1 dB budget, sequence by sequence. Over budget counts the sequences whose delivered loss exceeds the budget; the last two columns give the least-saving sequence at each rate. The negative entries are the 416×240 class, where a frame is two tiles.

5.2 Is per-tile adaptivity necessary?

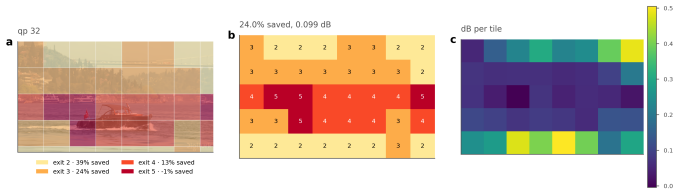

Figure 18. Where the decoder spends. Bosphorus at q32, a 0.1 dB budget. (a) the assignment overlaid on the frame, (b) the exit index per tile, (c) the quality each tile gives up, against its *own* full-depth reference. Water and sky leave at the shallowest exit; the boat and the shoreline run deep.

q	Allocation	ΔPSNR (dB)	MACs saved (%)
0	uniform, exit 2†	0.179	39.1
	uniform, exit 3	0.093	24.2
	uniform, exit 4	0.059	13.0
	uniform, exit 5	0.036	-1.0
	random (matched mix)	0.125	29.9
	rate-ranked (free)	0.107	29.9
	oracle	**0.100**	**29.9**
16	uniform, exit 2†	0.219	39.1
	uniform, exit 3†	0.118	24.2
	uniform, exit 4	0.075	13.0
	uniform, exit 5	0.045	-1.0
	random (matched mix)	0.133	25.6
	rate-ranked (free)	0.106	25.6
	oracle	**0.100**	**25.6**
32	uniform, exit 2†	0.282	39.1
	uniform, exit 3†	0.147	24.2
	uniform, exit 4	0.090	13.0
	uniform, exit 5	0.054	-1.0
	random (matched mix)	0.141	20.9
	rate-ranked (free)	0.107	20.9
	oracle	**0.100**	**20.9**
48	uniform, exit 2†	0.333	39.1
	uniform, exit 3†	0.176	24.2
	uniform, exit 4†	0.103	13.0
	uniform, exit 5	0.062	-1.0
	random (matched mix)	0.145	18.3
	rate-ranked (free)	0.107	18.3
	oracle	**0.100**	**18.3**
63	uniform, exit 2†	0.396	39.1
	uniform, exit 3†	0.206	24.2
	uniform, exit 4†	0.116	13.0
	uniform, exit 5	0.072	-1.0
	random (matched mix)	0.141	15.6
	rate-ranked (free)	0.110	15.6
	oracle	**0.100**	**15.6**

Table 10. Adaptivity against two controls at the 0.1 dB budget. † marks uniform depths whose distortion exceeds the budget, so they are not admissible allocations at all. “Random” draws each frame's map from the oracle's own exit histogram and shuffles it across tiles, which preserves the average cost and the mix of depths while discarding the content dependence.

The obvious alternative to routing tiles is to run every tile at the same shallower exit and accept the loss. The table puts that option, together with two stronger controls, at matched compute (Table 10).

Figure 18 shows what an allocation looks like on one frame. Read it against the picture rather than against the histogram: the water and the sky leave at the shallowest exit the ladder offers, the boat and the shoreline run to the deepest, and the split is 15-19-6 over the three rungs the multiplier used. What panel c adds is the price: no tile pays more than 0.31 dB against its own full-depth reference while the frame stays at 0.100 dB, which is the concentration this section measures, drawn on one picture.

The uniform rows answer the question directly, and the answer sharpens with rate. At the lowest rate a static decoder can reach exit 3 inside the budget and save 24.2%, against the Lagrangian oracle's 29.2%. At q48 and q63 the only uniform depth that fits is the deepest one, which *costs* 1.0% instead of saving anything. At those two rates, then, the comparison is between 18.3% and 15.6% against nothing at all, not against something smaller. Averaged over rates, the best static allocation saves 9.7% where the adaptive one saves 21.5%.

The two shuffled rows separate effects that are easy to conflate. We keep the oracle's own exit histogram, so the mix of depths and hence the average cost are unchanged, and assign it to tiles at random. Quality falls sharply. What the allocation buys is knowing *which* tiles can afford to run shallower; running some fraction of them shallower is worth nothing by itself. The oracle-histogram bit ranking keeps the same histogram and orders it by a signal the decoder already holds, namely the bits the entropy model spent on each tile. This is the cheapest router we can think of. It has no parameters and no training, and the number is available before the trunk starts. It is also the natural analogue of the confidence rules used by early-exit classifiers [3, 20], which likewise read a signal off the network's own output.

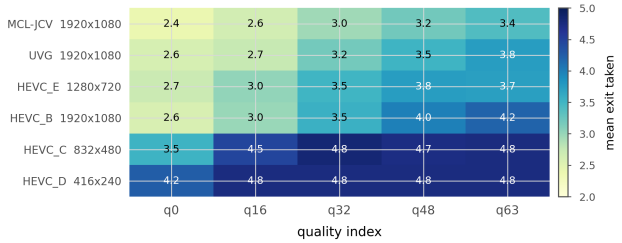

Figure 19. How deep each class has to go. Mean exit taken by the tiles of each test class at the 0.1 dB budget, ordered by that mean. Resolution is the strongest predictor: the 416×240 class has two tiles a frame and almost no choice, and the 1080p classes have forty and use the whole ladder. This is the dependence Section 5.3 measures, seen per class rather than pooled.

It recovers most of the gap. At q63, random costs 0.141 dB and the oracle 0.100 dB at identical compute, while the oracle-histogram bit ranking costs 0.110 dB. That is 75% of the oracle's advantage over chance, at 0.68–0.79 agreement with the oracle's map. A learned router therefore has to beat a calibrated bit rule that is already three quarters of the way there. We would not have run that comparison without this control, and we suggest that any adaptive-inference paper report it. Given the Lagrangian oracle's histogram, what we measure here is *ranking* and nothing else. Section 5.6 removes that crutch and turns the same signal into a complete routing rule, which matches the trained head across the whole rate range.

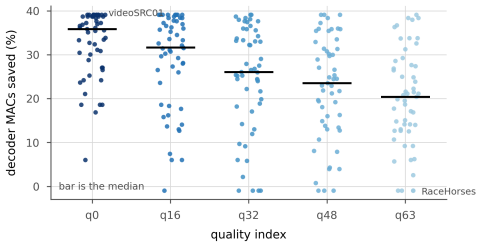

Figure 20. What the set mean hides. Every test sequence as a point, at the 0.1 dB budget; the bar is the median. At the lowest rate the saving runs from 6.0% to 39.1% across the 53 sequences and at the highest from -1.0% to 39.1%. The spread within a rate is larger than the difference between rates, and it is content, not noise: the same sequences sit at the same end of it at every rate.

5.3 Resolution, and the granularity of a tile

This comparison is indicative and confounded. The 128 and 256 pixel ladders are different training runs, so anything that separates them separates two runs as well as two tile sizes. The size of that confound is measurable and it is not small: two runs of the SAME recipe, at a matched epoch on the same frames, differ by 4.8 saving points, which is as large as the tile-size effect reported below. We report the comparison because the resolution dependence it exposes is real and was hidden by a 1080p-only test set, and we do not read the tile size as its cause.

frame	tiles	exits taken	saved %	delivered dB	worst tile dB
Bosphorus, 1920×1080	40	e2:15 e3:19 e4:6	28.1	0.100	0.310
BasketballDrive, 1920×1080	40	e2:14 e3:13 e4:11 e5:2	25.1	0.098	0.232
FourPeople, 1280×720	15	e2:3 e3:4 e5:8	13.8	0.100	0.338
PartyScene, 832×480	8	e3:2 e4:4 e5:2	12.3	0.098	0.238
RaceHorses, 416×240	2	e4:2	13.0	0.097	0.113

Table 11. One frame from each class, at q32 and a 0.1 dB budget. The exits column is the histogram over the ladder, so e4:2 is two tiles at exit 4.

The table says where the method stops being adaptive (Table 11). Reading down it, the exit histogram narrows from a mixture over three rungs to a single one, and at 416×240 both tiles take exit 4 and the frame simply receives whatever that rung costs. That is not a router failing. It is a frame with two tiles and no allocation to make, and it is why the low-resolution classes sit at the bottom of every per-class result in this paper. The worst-tile column moves the other way and shows the price the budget is paying somewhere in the frame: 0.310 dB on one Bosphorus tile against a 0.100 dB frame average, which is the concentration Section 5.5 measures.

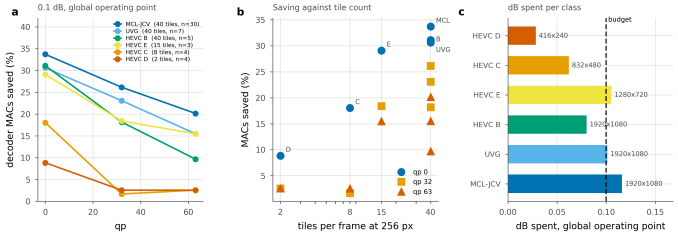

Figure 21. Saving by test class at one global operating point. λ is bisected once so the whole set lands on the budget, exactly as it would be in deployment; each class is then reported at that λ. The ordering follows tile count, not content.

Class	Resolution	Tiles	n	q0	q32	q63
MCL-JCV	1920x1080	40	30	33.7	26.2	20.2
UVG	1920x1080	40	7	30.6	23.1	15.5
HEVC_B	1920x1080	40	5	31.1	18.2	9.7
HEVC_E	1280x720	15	3	29.1	18.4	15.5
HEVC_C	832x480	8	4	18.0	1.7	2.5
HEVC_D	416x240	2	4	8.8	2.5	2.5

Table 12. Saving by test class (%), at a 0.1 dB budget set globally over all 53 sequences rather than per class. “Tiles” is how many 256 px tiles a frame of that resolution contains. The saving tracks tile count much more closely than it tracks content.

Completing the test set exposed a dependence that the 1080p-only subset had hidden. The table groups the saving by class (Figure 21) (Table 12). At 1080p, where a frame is 40 tiles, we save 33.7% (MCL-JCV), 30.6% (UVG) and 31.1% (HEVC B) at the lowest rate, three classes of very different content within three points of each other. At 832×480 a frame is 8 tiles and the saving is 18.0%; at 416×240 it is 2 tiles and 8.8%. At high rate the small resolutions fall to 2.5% against 20.2% for 1080p.

The natural explanation is granularity. With two tiles per frame there is almost no allocation left to make, and the Lagrangian degenerates towards a uniform choice. That suggests a fix. Choose the tile size relative to the frame, since the MAC count does not depend on tile size at all and only the seam does.

We tested that fix and it mostly does not work. We compare the 256 px tiling against a 128 px one at q0, which multiplies the tile count by 3.4. MCL-JCV (1080p) goes from 36.3 to 34.3, HEVC B (1080p) from 33.8 to 32.1, HEVC C (832×480) from 20.9 to 17.6, and HEVC D (416×240) from 11.3 to **13.7**.

Smaller tiles help only at the smallest resolution, where the count goes from two to eight. Everywhere else they cost between 1.7 and 3.3 points, including at 832×480, where the count rises from 8 to 28 and the saving

still falls. Beyond a modest number of tiles, the extra seam outweighs the extra granularity. We report the comparison as indicative, since the two tile sizes come from different training runs and tile size is therefore confounded with training. But the direction is consistent, and it is enough for us to say that a resolution-adaptive tile size is not the easy win the granularity argument suggests (Figure 19).

5.4 The band a distortion budget works in

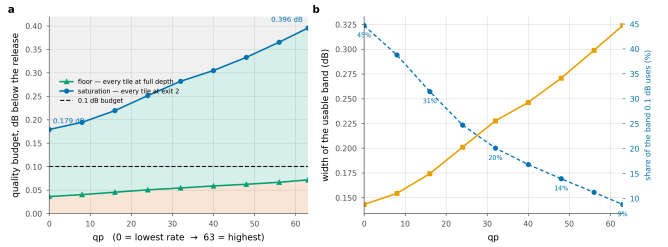

Figure 22. Three regions, and only the middle one is a design choice. Below the floor no allocation meets the budget; above saturation every tile already takes the cheapest exit. The 0.1 dB budget uses 45% of the band at the lowest rate and 9% at the highest.

q	0	16	32	48	63
Floor D_min	0.036	0.045	0.054	0.062	0.072
Saturation D_sat	0.179	0.219	0.282	0.333	0.396
Usable band	0.143	0.174	0.228	0.271	0.324
0.1 dB uses	45%	31%	20%	14%	9%

Table 13. Floor and saturation, dB below the released decoder. The floor is what tiling costs with no early exit at all; saturation is where every tile reaches exit j and the ceiling is attained.

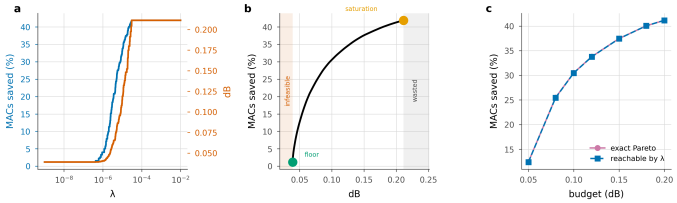

Figure 23. The structure of the allocation, on one frame. **a**, compute and distortion are monotone in the multiplier. **b**, the frontier, with both ends and both dead bands marked on the panel. **c**, what the Lagrangian reaches against the exact Pareto set, enumerated by dynamic programming.

The argument that follows assumes tiles are independent. It treats the frame's distortion as a sum over tiles, while the deblocking pass runs on the stitched frame and therefore sees the whole exit map at once, so the tiles are not strictly independent. We measured the size of that coupling over 60 allocations and the largest mismatch between the separable prediction and a real decode is 5.5×10^{-5} dB, four orders of magnitude below the budget. We proceed as if the objective were separable and treat that as measured rather than assumed.

The construction has enough structure to state as propositions, and we check each one numerically instead of asserting it (scripts/verify_theory.py, seven of seven).

1	the allocation decouples per tile
2	compute is non-increasing and distortion non-decreasing in λ
3	the sweep traces the lower convex hull, so it cannot reach an interior point of the achievable set
4	λ = 0 reaches the least distortion the ladder can produce
5	beyond a finite λ, computable in closed form, the allocation is the constant map to exit j, at cost exactly c_j
6	the ceiling is a function of the split depth alone

The third of those has a practical edge. The sweep reaches only hull vertices, so a budget that falls between two of them cannot be met exactly. We measured what that costs by enumerating the *exact* Pareto set with dynamic programming over tiles, which is feasible because there are only four distinct exit costs. The sweep reaches 95 allocations against a Pareto set of 635, and the loss from convexity is at most 0.05 saving points (Figure 23). For this problem the standard construction is essentially optimal.

It matters *why* that loss is small, since the reason is structural and points at what would make it smaller. The reachable costs form a lattice, and its spacing is set by how much the mean cost moves when the multiplier crosses a breakpoint. If m tiles each move to the next exit there, the spacing is at most $m\cdot\max(c_{k+1} - c_k)/T$ saving points. The convexity loss cannot exceed the largest spacing, because a budget between two levels is served by the lower one. Here m=2, since tiles with identical rows switch together, and the bound is 0.745 points; the measured spacing sits exactly on it. Nothing in the expression depends on content or rate, and the bound falls as 1/T: halving the tile side puts four times as many tiles on the lattice, each carrying a quarter of the step. Fine granularity is what makes the Lagrangian relaxation lossless in practice, and no property of the images is doing that work.

Two numbers bound what any budget can do (Table 13) and Figure 22. The **floor** is the distortion of a tiled decode with every tile at full depth, which is pure tiling penalty: 0.036 dB at q0, rising to 0.072 dB at q63. A budget below it admits no allocation. The **saturation** point is the distortion when every tile takes exit j; any budget at or above it reaches the ceiling, and a larger one reaches nothing more.

One consequence of that is worth stating plainly. At q0 the ceiling is reached at 0.179 dB, so the architectural bound behaves as an operating point rather than as an asymptote. Past it, the limiting factor is the *ladder* and not the budget, which is an argument for a finer ladder before a looser budget. The same two numbers pick out a narrow regime: budgets in [0.179, 0.194) dB saturate the lowest rate and nothing else, a range 15 thousandths of a decibel wide.

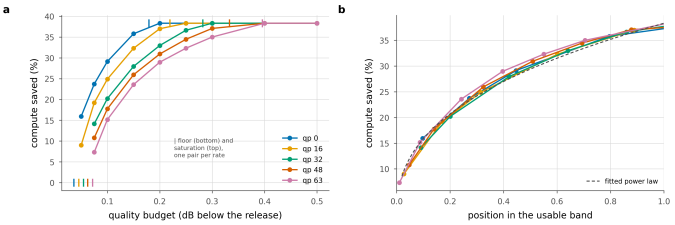

Figure 24. The band is the rate dependence. a Saving against the budget, with each rate's floor and saturation point ticked. **b** The same with the budget axis rescaled onto each rate's own band. Five curves become one; dashed is the one-parameter power law fitted to all of them.

And the band is almost all of the rate dependence. At a matched decibel the five rates are 14.0 points apart. Rescale the budget axis onto each rate's own band, with the floor at 0 and saturation at 1 (Figure 24), and they collapse onto a single master curve, 1.7 points apart on average and 2.1 at worst. The answer to "how much does a 0.1 dB budget buy at this rate" is therefore, to within a couple of points, "where does 0.1 dB sit in this rate's band". The floor and the saturation point are both in closed form and both cheap to measure. What they leave over is small enough that a deployment could calibrate the two ends and read the rest off one curve. That curve is a one-parameter power law, saving $\approx C\cdot u^{0.38}$, with C the architectural ceiling and u the position in the band. We fit it in log space over all five rates and get $R^2 = 0.989$, worst residual 2.1 points. The claim we make is the *rescaling*: measuring the budget in units of each rate's own band is what collapses the curves. The exponent is an empirical fit to that collapse and not a law. Inverse fits to the same frontier disagree in it by up to 40%, so we report it as a description of these curves and read nothing into its value.

The collapse appears robust across two checkpoints. We repeated the measurement on a different training run, BEST, a separate recipe taken at a different epoch, and the same thing happens. The rates are 16.9 points apart at a matched decibel and 1.6 after rescaling, and the

collapsed curve is again a power law, $R^2 = 0.984$. The *exponent* does not carry over: 0.33 there against 0.38 here, so it describes a trained decoder and not the architecture. What replicates is the collapse. Within a checkpoint, the rate dependence of the trade-off is the rate dependence of the band (Figure 25).

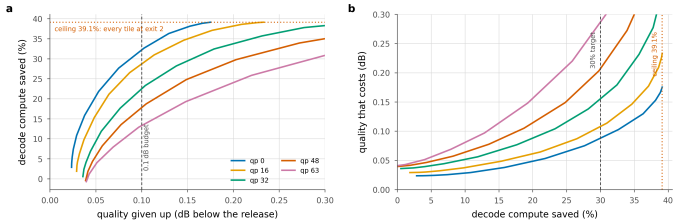

Figure 25. The trade-off, whole. a What a budget buys, per rate; the dotted line is the arithmetic ceiling, 39.1%, and q0 reaches it at 0.179 dB. **b** The same relation inverted, so it reads as what a saving target costs. The three budgets reported elsewhere are three points on this curve.

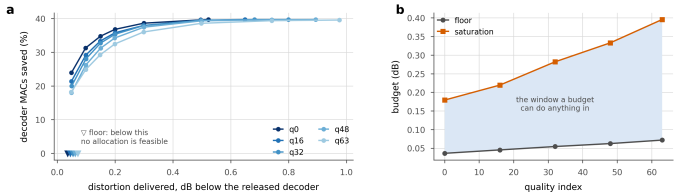

Figure 26. The window a budget works in. a, saving against the distortion actually delivered, per rate. Every curve begins at a floor, below which no allocation meets the budget, and flattens at a saturation point, above which a looser budget buys nothing. **b**, those two limits against rate. The shaded band is the only region in which a budget is a design choice rather than a formality.

5.5 Signalled versus predicted allocation

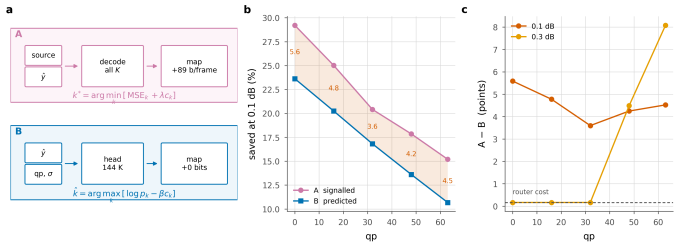

Figure 27. Who decides. a, the two decision paths side by side: the encoder's search over all K exits per tile against the head's forward pass over decoded data. **b**, saving at 0.1 dB; shading is what a bit-exact bitstream costs. **c**, that cost is smallest at q32\ and grows in both directions.

	0.1 dB	0.3 dB				
q	A	B	gap	A	B	gap
0	29.2	23.6	**5.6**	38.3	38.2	**0.2**
16	25.0	20.3	**4.8**	38.3	38.2	**0.2**
32	20.4	16.8	**3.6**	38.3	38.2	**0.2**
48	17.9	13.6	**4.2**	37.1	32.6	**4.5**
63	15.2	10.7	**4.5**	35.0	26.9	**8.1**

Table 14. Signalled against predicted at two budgets, same checkpoint and test set, with one router trained against the oracle on the deployed table at a single λ.

The table above compares the two, and what it prices is exactness against bits (Table 14, Figure 27).

Not signalling costs 3.6–5.6 points, roughly flat across rate, for zero added bits and a byte-identical file. The gap is smallest at q32 and widens toward both ends of the rate range.

It tracks |β|, the cost multiplier the bisection has to apply to move a router trained at one λ onto another operating point. At q32 the required β is 10, essentially none, since that is where the training λ lands, and there the gap is at its minimum. At q0 it is 146 and the gap is 5.6. A large β in either direction lets the cost term dominate the logits, which discards the content ranking the router learned. What we ship is one head for every rate, with β read from an offline table indexed by quality index and budget (Section 3.5). A head per operating point would close this gap and we do not propose it: it multiplies the stored model by the number of rates, and the rule of Section 5.6 closes more of the gap for nothing.

Loosening the budget closes the gap only where the budget saturates the ladder. At 0.3 dB the gap is 0.2 points at the three lowest rates. That is exactly the router's own 0.163% of decode, so its *prediction* is free there; both configurations send every tile to the cheapest exit, and nothing is left to predict wrongly. At the two highest rates 0.3 dB does not saturate, and the gap *widens* to 8.1 points. A looser budget gives the allocation more room to be wrong in as well as more room to be right.

For a system that trains once, the single-router number is the honest one, so that is what we report. It understates what B can do.

	β held out	β bisected on the test set					
q	β	saving	dB	β	saving	dB	at equal dB
0	146	23.6	0.098	146	23.6	0.099	-0.0
16	133	21.3	0.105	100	20.3	0.100	+-0.0
32	38	18.1	0.109	10	16.8	0.100	+-0.0
48	-34	14.9	0.108	-46	13.6	0.100	+0.0
63	–	–	–	-57	10.7	0.100	–

Table 15. Choosing β without the test set. Left, β bisected on the 512 held-out validation frames and then applied to the test frames unchanged; right, β bisected on the test frames themselves, which is what the signalled-against-predicted table reports. Savings are hook counts on the routed decode with the router's own compute charged against them. The last column re-bisects on the test set to the quality the held-out β delivered, so the two allocations are differenced at one distortion rather than across two.

Where the β above came from. Every β in the signalled-against-predicted table was bisected against the budget on the test frames themselves, which is the one thing Section 3.4 says a decoder cannot do. The table immediately above repeats the measurement with the table a deployment would ship (Table 15): β bisected once per quality index on the 512 held-out Open Images validation frames, then applied to the CTC frames without being touched again. Checkpoint, head, frames and budget are identical on both sides, so what separates the two columns is where β came from.

At q0 the two agree, 146 against 146 and 23.6% against 23.6%, which is the same measurement to a hundredth of a point. At the rates in between, the held-out β misses the budget on the high side: it delivers 0.109 dB at q32 where 0.1 dB was asked for, and 3 of the 4 rates it covers overshoot. Distortion and saving move together, so those rows also report more saving, and reading one column straight against the other would credit the router with compute it bought using quality the budget did not allow. The last column takes that back out, re-bisecting on the test set to the quality the held-out β actually delivered: at equal delivered quality the two allocations agree to within 0.0 points. What moves between the two sets is the decibel a given β delivers.

The highest rate fails harder than that. On the calibration frames the floor, the distortion tiling costs with every tile already at the deepest exit, is 0.104 dB at q63. That is above the budget, so no allocation on that set meets 0.1 dB and the bisection has nothing to return. The shipped table has a hole where the highest rate should be, and a decoder holding it falls back to the deepest allocation, which is our own full-depth path and costs slightly more than the release, so it saves nothing. The test frames, whose floor at that rate is 0.072 dB, would have supported 10.7 points. What fails there is the calibration set rather than the router: a budget written as an absolute decibel sits a different distance above the floor on 512px photographs than on 1080p video, and at the highest rate it sits below it.

We keep the test-bisected figures as B's headline, since the comparison against A is built on them, and the held-out table is the correction a reader should apply. Calibrating on frames whose tiling penalty matches the ones the decoder will meet is the fix, and for a video decoder that means calibrating on video.

Retraining with the mask fixed raises agreement and does not simply raise the saving. The head above was trained against a suppression term sitting above its own logits (below). We retrained it with the mask fixed, same recipe and same λ. Held-out agreement rises from 0.718 to 0.808, and the deployed saving moves by +3.1 points at q0 and −3.4 at q63. We find the retrained head ahead where the required β is small and behind where it is large, which is the same axis the gap runs along. That is the second time in this paper that agreement with the oracle has moved opposite to the quantity that matters. We keep the original head for every other measurement so the comparisons stay on one system, and we read the retrain as evidence that the mask cost the head real capacity, and that agreement is not the objective.

An earlier version of this measurement put the gap at 1.7 points at q0, rising to 13.3 at q63. The exit mask was at fault. It suppresses exits below the split depth by assigning -1e4, the head's own logits had drifted to that scale, and the suppressed entries were therefore the largest in every row. A large share of every tile went to the cheapest exit for a reason unrelated to its content. The mask is now negative infinity. Fixing it costs 3.5 points at q0, where the accident happened to agree with the oracle, and buys 9.4 at q63, where it did not.

What each router input is worth

The head reads four per-tile signals and the quality index, and one of them carries almost all of it. We retrain the same head once per input group, zeroing the others after their projection rather than deleting them, so architecture, parameter count, optimiser, seed and data order are identical across the six runs and only what the head is allowed to know differs. The quality index stays live everywhere: it is one number per frame and cannot tell two tiles of a frame apart, so the run with it alone is the floor that no per-tile information reaches, 0.4923 agreement.

live inputs	agreement	pm s.e.	last 250	over floor
stem,qp	0.7750	0.0074	0.7607	+0.2744
stem,latent,scales,bits,qp	0.7721	0.0075	0.7543	+0.2715
latent,qp	0.7448	0.0080	0.7433	+0.2442
scales,qp	0.7156	0.0083	0.7170	+0.2150
bits,qp	0.6202	0.0091	0.6137	+0.1196
qp	0.4923	0.0114	0.4900	-0.0083

Table 16. What each router input is worth. Agreement with the search's exit choice on held-out frames. Read the first column against the last: only the stem moves agreement away from what a single constant exit already achieves.

The stem alone reaches (Table 16) 0.7750, +0.2744 over the floor, and adding the latent, the scales and the bit count on top of it reaches 0.7721, which is not an improvement. The signals are not independent: the entropy model's scales and the bits spent are both functions of the same latent the stem was computed from, so the head is being given one piece of information in four dressings. That is worth stating beside Section 5.6, where a rule reading the bit count alone beats the whole head: the bit count is not a weak input, it is the same input, read by something that does not have to learn what to do with it.

5.6 A router with no parameters

Before a 144 K head is worth its 0.163% of the decode, it has to beat what the decoder already knows (Figure 28, Table 17). The entropy model has produced one number per tile before the trunk runs, and at no cost: how many bits that tile's latents took.

We turn it into a routing rule with no learned parameters. We model the per-tile distortion as rank-1 in the log domain

$$\log D(t,k) \approx \alpha \log b(t) + c + \log \varphi_k \qquad (7)$$

where b is the tile's bit count normalised by the frame mean and ϕ is a K-vector saying what each exit costs on an average tile. We fit (α, c, ϕ) by least squares, leave-one-sequence-out, so that no sequence contributes to the profile that routes it. The same Lagrangian the oracle runs is then run on the surrogate. Nothing is signalled and nothing is trained; the arithmetic is one scalar per tile.

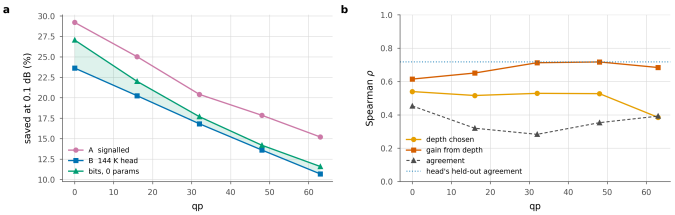

Figure 28. The calibrated bit rule. a Saving at 0.1 dB; shaded where the calibrated bit rule beats the trained head. **b** What a tile's bit count is correlated with, against how often the rule agrees with the oracle outright; dotted is the head's own held-out agreement.

q	rate-rank	B router	A signalled	ρ depth	ρ spread
0	**27.1**	23.6	29.2	+0.54	+0.61
16	**22.0**	20.3	25.0	+0.52	+0.65
32	**17.7**	16.8	20.4	+0.53	+0.71
48	**14.2**	13.6	17.9	+0.53	+0.72
63	**11.6**	10.7	15.2	+0.39	+0.68

Table 17. The calibrated bit rule, 0.1 dB, same test set; it is the "rate-rank" column. Bold where the rule beats the trained head. ρ are Spearman correlations between a tile's bit count and, respectively, the depth the oracle assigns it and the distortion it stands to gain from that depth.

Figure 29. What 28.1% of the arithmetic costs, to look at. Bosphorus at q32, one frame, with λ bisected on that frame to 0.100 dB. Both crops are decoded from the *same* latent at 0.2743 bpp: the released decoder reaches 39.48 dB and the routed decode 39.39 dB. The crop is centred on the tile that gave up the most, tile 18 of 40, not on a flattering one. Right, the absolute difference at ×20.

It beats the trained head at every rate. The zero-learned-parameter rule is ahead of the 144 K router at all 5 measured rates, by margins that run from half a point at q48 to 3.4 points at q0. A head trained on this decoder against this oracle therefore returns nothing over a rule with no parameters at all, and it carries 0.163% of the decode that the calibrated bit rule does not.

Why it works is not that it agrees with the oracle. It agrees on 0.28–0.45 of tiles, well below the router's 0.718, and still saves more at every rate. Agreement weighs a disagreement on a tile where two exits are within a hair of each other exactly as heavily as one where the choice is most of the frame's error, and most tiles are the former. The ordering is what the rule gets right. Bits correlate with the *spread* across the ladder, that is, with how much a tile stands to gain from depth, at ρ_spread = 0.61–0.72 at every rate.

At a looser budget it stops being a baseline and becomes the answer. At 0.3 dB the calibrated bit rule matches the oracle exactly at the three lowest rates, where both reach the same architectural ceiling, comes within 0.2 points of it at q48, and gives up 34.3% against the oracle's 35.0% at q63. That is ahead of the trained head at every rate, by up to 7.4 points. The head is charged for itself and the rule is not. At a loose budget the head's ordering also has less left to contribute, because once the budget saturates the ladder there is little ordering left to get right, and the rule gets it. At 0.5 dB it reaches the ceiling at every rate and its assignment is *identical* to the oracle's on every tile.

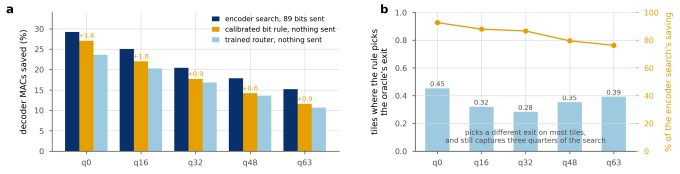

Figure 30. Three ways to choose an exit. a, saving at 0.1 dB. The encoder search sees the source and sends 89 bits a frame; the other two send nothing. The bit rule beats the trained head at every rate, with no learned parameters against the head's 144,030. **b**, bars are the tiles where the rule picks the search's exit, the line the fraction of its saving it captures anyway.

What it cannot do is see anything beyond that ordering. A rank-1 model in the level assigns every tile the same relative profile over exits, so b only decides where on the ladder a tile falls, never the shape of its trade-off. That is the ceiling this baseline sits at, and it is the part a learned head should be earning its parameters on. Ours does not earn them at any rate we measured.

q	A signalled	B router	bits, 0 params
0	34.0	31.6	30.9
16	27.7	25.1	**27.1**
32	22.3	16.5	**22.4**
48	19.8	13.3	**19.1**
63	17.2	9.0	**16.2**

Table 18. The same comparison on a second training run (BEST), 0.1 dB, same test set. Bold where the calibrated bit rule beats that run's own trained head.

It replicates on a second training run. BEST is a separate recipe at a different epoch, with its own router trained the same way. There the calibrated bit rule beats the trained head at (Table 18) 4 of 5 rates, by up to 7.2 points, and comes within 3.1 points of the *oracle* everywhere. The margins there are wider than here, and the one rate it concedes is the only rate on either checkpoint where the head finishes ahead, by under a point. We do not read that as showing a learned head cannot be worth its parameters, only that neither of our training runs produced one that is. The calibrated bit rule stays close to the oracle on both.

q	B	5%	10%	20%	50%	A
bits/frame	0	14	26	44	83	100
0	24.4	25.1	26.0	27.3	30.1	29.9
16	20.9	21.7	22.5	23.7	25.7	25.6
32	17.4	18.1	18.8	19.8	21.1	20.9
48	14.1	14.9	15.8	16.8	18.3	18.3
63	11.1	11.9	12.9	13.9	15.3	15.6

Table 19. Configuration C at 0.1 dB. Columns are the fraction of tiles the encoder overrides; the two end columns reproduce B and A to within 0.1 points, which is the check that the interpolation is real.

Per-block bit allocation is a standard quantity in learned compression, where it is something to *choose*; block-level rate control sets it so that complex regions get more bits [42]. We read the same number in the other direction, after the fact and at the decoder, as a statement about how hard a region was. It costs nothing because someone else has already paid for it.

q	bits	0.1	0.25	0.5	0.75	0.9	head
0	**29.8**	27.9	27.9	27.9	28.0	27.9	28.0
16	**24.1**	23.3	23.0	23.0	22.8	22.7	22.8
32	**19.6**	19.5	19.4	19.3	19.3	19.3	19.3
48	14.9	16.9	**17.0**	16.8	16.7	16.7	16.6
63	12.4	**13.2**	12.4	11.7	11.6	11.4	11.4

Table 20. Blending the two decoder-side signals at 0.1 dB, both normalised to unit mean, weight w from the calibrated bit rule to the head's ordering. Bold is the best per rate.

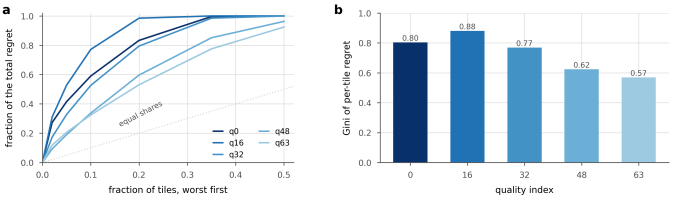

Figure 31. The loss is concentrated. a, the share of the total regret carried by the worst fraction of tiles, per rate; the dotted line is what an even spread would look like. **b**, the same as a Gini coefficient. At every rate a small minority of tiles carries most of what staying silent costs, which is why signalling a fraction of the map recovers most of the gap.

Are the two signals complementary? Barely. We normalise both surrogates to unit mean and blend them with one weight w, where w=0 is the calibrated bit rule and w=1 the head's ordering. The best blend is w=0 at the three lowest rates (Table 20) and a small head weight at the two highest, worth +2.1 points at q48 and +0.9 at q63. The head reads the entropy model's scales, so it already has most of what the bit count carries; what it adds is confined to q48 and q63.

How much a trained head varies, and what that does to the claim. No head here was trained twice at two seeds, so we have no seed variance to report. What we can report is the spread between two heads trained independently on the same decoder, which is looser than seed variance and larger. The joint and the frozen head differ by +3.2 points at the lowest rate and -5.6 at the highest, a range of 8.8 points, and they cross: whichever is better depends on the rate. At some rates that spread is wider than the rule's margin over either of them. So the claim we make is not that no learned router can beat the bit rule. It is that a rule costing nothing sits inside the band two of our own trained heads span, which is where a control belongs.

A learned component should be measured against the free alternative, and it rarely is. In adaptive inference the usual controls are a uniform allocation and a random one. Both are much weaker than a decoder-side signal that is already lying around (Figure 30).

5.7 Signalling only what the router gets wrong

The encoder knows tile by tile where the router will be wrong, because it can run that router itself (Section 3.1), and nothing obliges it to correct every one of them. That is the room configuration C works in (Figure 32, Table 19).

We choose the ρN overridden tiles by Lagrangian regret, Δ(t) = L(t,k-hat) − L(t,k*) with L(t,k) = D(t,k) + λc_k the objective of the Lagrangian in Section 3.5, so Δ(t) is exactly what that objective loses on tile t by staying silent. We hold β at B's value and bisect λ over the overrides to land back on the budget. The map costs an entropy-coded mask, N·H(ρ) bits with H the binary entropy, plus 3 per override.

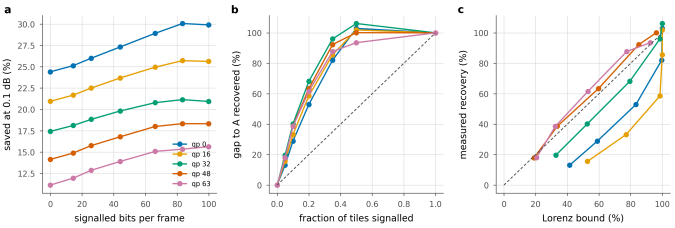

Figure 32. Partial signalling. a, saving against the bits spent on the map; each line runs from B on the left to A on the right. **b**, the same normalised by the A-to-B gap; dashed is proportional recovery. **c**, against the fixed-λ Lorenz bound; points above the dashed line are allocations one multiplier cannot reach.

The two ends check out. At ρ=0 the sweep reproduces configuration B and at ρ=1 it reproduces A, each to within 0.43–0.78 points. That residue is not disagreement: this sweep was never hook-counted, so its savings come from the arithmetic model, and the model over-reports by exactly that much. Neither end is imposed; both fall out of the same code path run against independently measured files, so the columns between them

are measuring something real.

Most of the gap is cheap. Overriding a tenth of the tiles for 26 bits per frame recovers 29–39% of the gap, and a fifth recovers 53–62%. Recovery is concave everywhere, so the first bits spent are the most useful ones.

And a partial map can beat a complete one. At 4 of 5 rates, signalling half the tiles for 83 bits saves *more* than signalling all of them, by up to 0.21 points, at the same distortion. Both land within the bisection's 5e-4 dB tolerance of the budget, and the local exchange rate is about 48 saving points per decibel, so the tolerance is worth 0.02 points against a margin ten to twenty times that. The margin is therefore real, and it does not come from a better predictor. The hybrid has *two* multipliers where A has one: the oracle's λ governs the overridden tiles and the router's fixed β the rest. A two-multiplier allocation reaches points on the distortion–compute plane that a single Lagrangian sweep cannot, for the same reason the sweep misses interior Pareto points at all. Configuration A is optimal among allocations reachable by one multiplier, and that is a smaller set than the word suggests.

Why the recovery is concave, and what bounds it. At a fixed λ the objective is separable, so overriding a set removes *exactly* the sum of that set's regrets. The recovery of the *objective* is then the Lorenz curve of the per-tile regret distribution, and its curvature is that distribution's Gini coefficient, 0.49–0.73 across rates. What the table reports is saving at fixed distortion, not the objective, and returning to the budget means re-bisecting λ. That re-bisection is also what lets the two-multiplier allocations above escape the bound.

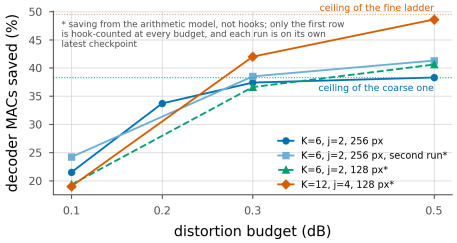

Figure 33. The right ladder depends on the budget. Saving against budget for four ladders, with each one's architectural ceiling as a dotted line. A finer ladder has a higher ceiling and a higher floor, so the curves cross: the coarse ladder wins where the budget is tight and cannot be spent, and the fine one wins where it is loose enough to reach exits the coarse ladder does not have. The crossing is the design choice this section is about.

At the loose budget it matters where the gap is. At 0.3 dB the three lowest rates are saturated, and there is nothing for a partial map to buy. At $q48$ and $q63$, where the gap is 8.1 points, half the map recovers 68–91% of it. The allocation follows the same rule here as everywhere else, which is to spend the bits where the ladder still has somewhere to go.

A better predictor concentrates its regret, which is what the Gini was for. We rebuild C on the retrained head of Section 5.5. The Gini of the per-tile regret rises at 4 of 5 rates, to 0.68–0.91. Its mistakes are rarer and larger. The same table shows the consequence. C converges on A faster from a better base and has less left to recover, and the half-beats-all effect disappears at q0, because a base close to A leaves little for the extra multiplier to exploit. The Lorenz reading is therefore worth having as a diagnostic and not as a decoration; it reports what *kind* of predictor one has as well as how good it is.

Which predictor should C be built on? Either will do, and the answer follows the same split as Section 5.6. Running the identical override rule over the calibrated bit rule instead of the head is worth up to 3.0 points at the lowest rate and costs up to 0.2 at the highest. Both converge on A at ρ=1 to the second decimal at every rate, which is the third independent check that the two code paths agree. The best single operating point we measured is the calibrated bit rule with half the map signalled, at 30.5% at q0. That is 0.61 points *above* full signalling, with no learned component anywhere in the decoder.

Configuration C is what we would ship where a small map is tolerable and a large one is not. It also reframes the A–B gap. What the gap measures is the price of *silence*, charged tile by tile, rather than the price of prediction.

5.8 Does the map have to be recomputed?

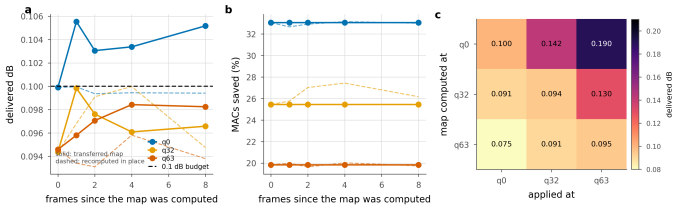

Figure 34. Reusing an exit map. Solid is the transferred map, dashed the one recomputed in place. **a**, **b**: reuse across frames of the same sequence. **c**: reuse across quality index; the entry is the delivered distortion against a 0.1 dB budget.

How much A's extra decode at the encoder matters depends on how often the map has to be recomputed, and we have not seen that measured anywhere (Figure 34).

Across frames. Reuse is close to free. We take the map found on frame 0 and apply it eight frames later; at q0 the delivered distortion rises by 0.005 dB, five percent of the budget, and the saving does not move at all: 33.05% transferred against 32.66–33.15% recomputed. The allocation is a property of where the content is hard, and that moves slowly. A codec would search once per group of pictures instead of once per frame, and divide the encoder cost by the group length.

Across rate. Here the map does have to be recomputed, and the direction of the reuse decides how badly. Found at q0 and applied at q63, it delivers 0.190 dB against a 0.1 dB budget, claiming the low-rate saving of 33.05% while spending nearly twice the quality it is allowed. The reverse is safe but wasteful: the q63 map applied at q0 delivers 0.075 dB and only 19.8% where \30.6% was available. Shallow exits are cheap in quality at low rate and expensive at high rate, so a map is calibrated to the rate it was found at, and reusing one upward breaks the quality guarantee without anything in the decode reporting that it has. In our reuse probe, then, an encoder can search once per rate and reuse that search across the frames we tested. How far that carries beyond the offsets and sequences probed here we have not measured.

What the frame mean is made of

The budget binds on a frame, and a frame is 40 tiles. At the 0.1 dB budget the median tile of the lowest rate gives up 0.109 dB, which is about what the frame gives up, but the ninety-ninth percentile is 1.18 dB and the worst single tile in 1765 is 1.97. 26 tiles lose more than a decibel while their frames stay inside a tenth of one (Table 21).

q	tiles	median (dB)	p95 (dB)	p99 (dB)	max (dB)	over 1 dB
q0	1765	0.109	0.463	1.184	1.971	26
q16	1765	0.106	0.480	0.852	1.495	7
q32	1765	0.104	0.400	0.671	1.587	2
q48	1765	0.100	0.441	0.934	1.636	17
q63	1765	0.098	0.288	0.539	1.264	4

Table 21. The distribution behind the frame mean. Per-tile loss against the released decoder at the 0.1 dB budget, over every tile of every test sequence.

Which tiles they are is the part that matters, and it is the seam of Section 4 arriving in the allocation. Split the tiles by whether their receptive field reaches an invented value: the interior tiles give up 0.121 dB on average and the 547 tiles with padding give up 0.260, 2.2 times as much. The border is not only where the picture is damaged, it is where the allocation is most expensive, because a tile that already carries a seam has less room left inside the budget before it has to decode deeper. A

deployment that wants a per-tile guarantee rather than a per-frame one has a straightforward lever: clamp the shallowest exit a border tile may take, at the cost of the saving those tiles carry.

The tail also has a depth. Split the same tiles by the exit they took and the whole of it sits at the shallowest rung: tiles at e2 lose 0.202 dB on average and 79 of them lose more than half a decibel, while every tile at a deeper exit averages 0.085 dB and 0 of them pass half a decibel. The multiplier is doing what it was asked to do, spending the budget where it is cheapest, and the consequence is that the loss is not spread thinly over the frame but concentrated on the tiles it sent furthest. That is the same concentration Section 5.5 uses to build configuration C, seen from the distortion side rather than the regret side.

A budget in one metric is not a budget in another

Every allocation in this paper is bisected on PSNR, which prices a squared error and nothing else. The obvious question is what that does to a metric built to track perception, so we measure MS-SSIM on the same frames at the same operating point (Table 22), converted to decibels by $-10 \log(1 - m)$ so the two are read on one scale.

q	PSNR lost	MS-SSIM lost	released	routed
	(dB)	(dB)	(MS-SSIM)	(MS-SSIM)
q0	0.102	0.052	0.8993	0.8981
q16	0.100	0.045	0.9490	0.9485
q32	0.100	0.040	0.9756	0.9754
q48	0.103	0.038	0.9879	0.9878
q63	0.118	0.046	0.9942	0.9941

Table 22. Two metrics at one operating point. The 0.1 dB budget, 53 sequences, configuration A. MS-SSIM is converted to decibels by $-10 \log(1 - m)$. Both columns are what the routed decode gives up against the released decoder on the same latent.

The perceptual metric loses less than the budget spends. Against 0.105 dB of PSNR the same decodes give up 0.038–0.052 dB of MS-SSIM, 0.42 of the PSNR loss, at every rate. We do not read that as a perceptual claim. The shallow exits blur, and blurring costs a squared error more than it costs a structural similarity, so a budget written in PSNR is the conservative one of the two to write. What it is not is evenly priced across content: the sequence that gives up most MS-SSIM is videoSRC03, which loses 0.0024 of it while saving 34.1% of the arithmetic, and it is an ordinary 1080p clip from the middle of the set rather than one of the hard cases. A deployment that cares about a particular metric should bisect on that metric; nothing in the method depends on which one it is.

5.9 Complexity and wall-clock

q	Released	Routed	Saved (%)	
	(ms)	sorted (ms)	measured	MACs
0	111	78	**29.1**	35.3
32	111	86	**22.4**	28.0
63	111	94	**15.6**	20.1

Table 23. Wall-clock, 1080p, median of 40 interleaved iterations on one NVIDIA RTX A6000, 300 W board limit, PyTorch 2.6.0 and CUDA 12.4, single precision, at the 0.1 dB operating point. “MACs” is what the arithmetic predicts; “measured” is the sorted per-tile loop.

A saving in multiply-accumulates is not a saving in time (Table 7). Tiling costs 3.6% before anything exits early, measured as the deepest-exit tiled decode against the released full-frame one at the same arithmetic and the same weights. On top of that, the routed decode realises 27.9% at q0 where its operations predict 35.3%. Four fifths of the predicted saving arrives. The missing fifth is the tiling overhead together with the bookkeeping of a shrinking active set, and a MAC count sees neither. The shortfall scales with the saving instead of sitting at a fixed offset: at q63 we measure 15.6% realised against 20.1% predicted.

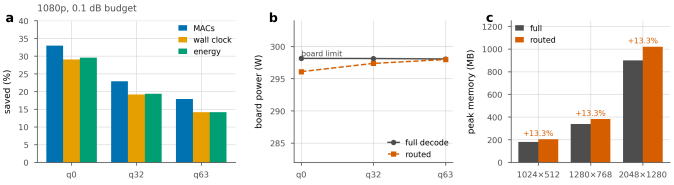

Figure 35. Three units for one saving. a, the same routed decode measured as arithmetic, as wall clock and as joules per frame, at 1080p and a 0.1 dB budget. Energy follows the clock, not the arithmetic. **b**, why: the board draws the same power either way, because the later groups run on a shrinking set of tiles and a partly idle GPU still draws its static power. **c**, peak memory, which routing raises rather than lowers, by 13.3% at every resolution that fitted on the card.

Seconds and joules. Multiply-accumulates are the right unit to optimise, because they do not depend on the machine, but they are not the unit a deployment cares about. We measured both on the padded 1080p frame, sampling board power from the driver at 100 ms and running each condition for a fixed wall time rather than a fixed iteration count, because the sensor updates too slowly for ten iterations of a 100 ms decode to mean anything. Idle power was measured before and after each condition so that a neighbouring process waking up would appear as drift rather than as a saving.

Energy follows time, not arithmetic (Figure 35). At q0, q32 and q63 the routed decode removes 33.0, 22.9 and 17.8% of the arithmetic; it removes 29.1, 19.1 and 14.1% of the wall clock and 29.6, 19.3 and 14.2% of the joules per frame. Board power barely moves, 298 W against 296 W at q0, because the later groups run on a shrinking set of tiles and a partly idle GPU still draws most of its static power. What early exit buys is a shorter decode, not a cooler one. Frame rate at 1080p goes from 9.0 to 11.2, and peak memory rises by 13.3% at every resolution we could measure: tiling turns one feature map into a batch of small ones, so the peak is set by the widest point of the ladder rather than by its deepest.

Sorting the tiles once by exit depth recovers part of it. The obvious implementation of a shrinking active set keeps a boolean mask, gathers the survivors and scatters the finished at every group boundary, and each of those boundaries forces a device-to-host synchronisation. Order the tiles by descending exit depth and “still active at group g” becomes a contiguous prefix: each group is then a slice instead of a gather, and one cumulative count supplies every boundary, so no per-group mask is built at all. The finished tiles come back through the inverse permutation in a single scatter. The arithmetic is unchanged and the output is bit-identical on GPU. At q0 the saving goes from 27.9% to 29.1%, a gain of (Table 23) 1.2 points, or a fifth of the shortfall, for a change that touches no arithmetic.

The same gap appears in our own accounting. The router is charged at its share of the decoder's multiply-accumulates, 0.163% (Section 3.1). We also timed the head directly, on the padded 2048×1280 frame the decoder actually sees, interleaved against a deepest-exit decode so that a co-tenant's load falls on both equally. It costs 0.50%, which is 3.0× what its arithmetic predicts, and for the same reason as everything else in this section: a 144 K-parameter head is launch overhead, and a MAC count cannot see a launch. Charged at measured time instead of at operations, every B number in this paper would fall by a further 0.33 points. We leave them charged at MACs because that is the convention the rest of the literature reports in, and we record the correction here so that it does not sit unstated.

A paper that quotes only operations would report 35.3% where the same code, run as written, delivers 29.1%. That is why we give the shortfall as well. The error runs one way: operations are an *optimistic* bound on this method, and the optimism grows with how much of the frame exits early.

Where the saving is realised, and where it is not

That optimism is a scheduling cost, and scheduling belongs to the device rather than to the method: the last groups of the trunk run on a handful of tiles and the card is largely idle waiting for them. If that is what the gap is, a machine that does not care how many tiles are in flight should not show it. We measured the same decodes on a CPU with 8 threads, and at three batch sizes on the GPU to see whether feeding it more work closes the gap instead (Table 24).

q	arithmetic	GPU, batch 1	GPU, batch 4	CPU
q0	34.6	29.8	31.0	36.3
q32	28.6	24.1	25.2	28.6
q63	22.3	17.9	19.1	18.0

Table 24. The same saving in three places. Per cent removed at the 0.1 dB budget on a padded 1080p frame: the arithmetic model, the GPU at two batch sizes, and the CPU. Arithmetic is the same in every column; only the machine changes.

On the CPU the bound is not optimistic. At the lowest rate the routed decode takes 5.6 seconds against 8.8, which is 36.3% removed where the arithmetic model predicts 34.6%. It realises slightly more than the operations count, because the tiles that exit early stop touching memory as well as arithmetic, and nothing there is waiting on a wide device to fill. The GPU recovers 1.1 points by batching, and stops there. At the highest rate the CPU falls back to 18.0%, in line with the GPU, because at that rate the allocation is deep and there is little left to skip. The claim we take from this is narrow: the gap between operations and time is a scheduling property of the accelerator, and on a device without one the operations count is the honest figure.

5.10 The right ladder depends on the budget

Ladder	Config	Ceiling	0.1 dB	0.2 dB	0.3 dB	0.5 dB
RECIPE512	K6 j2 256px	38.3	21.5	**33.7**	37.4	38.3
BEST	K6 j2 256px	38.3	**24.2†**	–	38.5†	41.3†
BEST128	K6 j2 128px	38.3	19.3†	–	36.6†	40.6†
FINE12	K12 j4 128px	49.5	19.0*†	–	**42.0†**	**48.6†**

Table 25. Ladder settings, mean saving (%) over the five rates, on each run's own latest checkpoint. Ceilings are 100(1-c_j) for that ladder, corrected by the offset the hook count shows. * some rate did not reach the budget, so the mean is over the rest. † the saving is from the arithmetic model rather than counted off the decode, and is therefore 0.4 to 0.8 points optimistic; only RECIPE512 has been re-measured with hooks at every budget.

Section 5.4 argued that once a budget saturates a ladder, the only way to spend more is an exit that does not exist (Figure 33, Table 25). The table measures that. A finer ladder (K=12, j=4) has a higher ceiling than the shipped one's 38.3%, and the two orderings cross somewhere between 0.1 and 0.3 dB. At 0.1 dB the coarse ladder wins, and the fine one cannot even reach the budget at the highest rate. Splitting later (j=4 against j=2) puts twice as many blocks in the per-tile section, which by the power law of Section 4 raises the floor. At 0.5 dB the fine ladder wins by 6.7 points, 48.6% against 38.3%, because the coarse one has been pinned at its ceiling since 0.3 dB and has nothing left to spend.

So the ladder is a choice made at the operating point, not a hyperparameter tuned once and fixed. The band of Section 5.4 tells you which side of the crossover a given budget sits on.

6. Against the released decoder

Everything above is measured against the released DCVC-UF intra decoder, but always as a percentage. This section states it once in the units a codec paper reports, so the comparison can be read without arithmetic.

The difference panel is the part worth reading (Figure 29). Amplified twenty times, what it shows is the rigging of the boat, the waterline and the edge of the flag, and almost nothing along the tile borders that cross the crop. The error the budget permits is spent where the picture is difficult, which is where the shallow exits fail, and not at the seams that Section 4 spends its length on. That is the seam repair and the border padding doing their work: the cut is still the largest single cost in the method, and by the time the reader sees the picture it is no longer the visible one.

The same frame at the top of the rate range says what the budget costs there (Figure 36). At q63, 0.4653 bpp, the released decoder reaches 45.26 dB and the routed decode 45.16 dB for 0.099 dB delivered, and the saving is 17.4% against 28.1% at q32. The allocation is what moved: on this frame 15 of 40 tiles take the shallowest exit at q32 and 1 does at q63, while the deepest exit goes from 0 tiles to 7. The picture is as hard to tell apart at either rate. What changes is how much of the decoder the budget will release, which is the rate dependence of Section 5.4 arriving in a single frame.

Figure 36. The same frame at the top of the rate range. Bosphorus at q63, 0.4653 bpp, the same 0.1 dB budget and the same crop as Figure 29. The released decoder reaches 45.26 dB and the routed decode 45.16 dB, for 17.4% of the arithmetic rather than 28.1%.

Decoder	GMAC /frame	kMAC /px	saved (%)	dB below release	BD-Rate (%)
released DCVC-UF intra	453.5	219	–	0.000	0.00
FLEX-UF, 0.1 dB budget	**355.8**	**172**	**21.5**	0.100	1.03
FLEX-UF, 0.3 dB budget	**283.8**	**137**	**37.4**	0.256	2.81
FLEX-UF, 0.5 dB budget	**279.7**	**135**	**38.3**	0.282	3.08

Table 26. FLEX-UF against the decoder it modifies. The released DCVC-UF intra decoder and the same decoder with the exit ladder, at the three budgets. Arithmetic is counted with hooks on the executed decode. “dB below release” is what the routed decode actually delivers, which is under the budget at 0.1 dB and well under it at 0.3 and 0.5, because by then the ladder has saturated and the budget cannot be spent. BD-Rate is the rate a codec would have to add to buy that quality back.

The released decoder spends (Table 26) 453.5 GMAC on a padded 1080p frame, which is 219 thousand multiply-accumulates for every pixel it produces, and it spends the same on every pixel whatever the picture is doing there. At a 0.1 dB budget the ladder takes that to 356 GMAC, 172 kMAC per pixel, and delivers 0.100 dB below the release for a BD-Rate cost of 1.03%.

The 0.3 and 0.5 dB rows are the same measurement with the constraint loosened, and they say something the percentages hide. Between them the saving moves by less than a point, from 37.4% to 38.3%, while the budget grows by two thirds. The ladder has run out of exits: at 0.3 dB most tiles are already on the cheapest rung they are allowed and the extra allowance buys nothing. The delivered distortion says the same, 0.256 dB against a 0.3 dB allowance. A looser budget is not the way to get more out of this decoder; a finer ladder is, and Section 5.10 measures one.

In wall clock on one RTX A6000 the 0.1 dB row decodes a padded 1080p frame in 89.6 ms against the release's 110.8, which is 9.0 frames per second becoming 11.2, and it does so for 26.6 J against 33.0. The arithmetic saving is 22.9% at that rate and the energy saving is 19.3%, and the gap between those two numbers is the scheduling cost of a decode that runs its last groups on a handful of tiles.

7. Limitations

That result is bounded on four sides: by a cost the method pays and cannot yet remove, by the kind of frame it was measured on, by where in

its training the measured model sits, and by the fact that there is one of it. We take them in that order.

The seam is reduced, and the exact remedy is not usable as it stands. The halo exchange removes 47–91% of the floor and is bit-exact at uniform depth (Section 4.4). It also destroys the routed saving, because routing puts neighbouring tiles at different depths. We do not know whether a decoder trained with the exchange can recover both at once, and that is the experiment we would run next. Until someone runs it, the floor is a cost this method pays.

Intra frames only. This is the image path of a video codec. Extending the ladder to inter frames raises a question we do not answer here, because an exit map propagates through the reference chain and a shallow tile in one frame is a worse reference for the next one.

The checkpoint is early in its schedule. Every number in this paper is measured on one pinned checkpoint, runs/RECIPE512/ckpt_PAPER.pth.tar, taken at the end of the first pass over the training set. It is pinned because a moving checkpoint was overwritten mid-measurement once, and a paper whose tables come from three different epochs is worse than one whose tables are early. Later checkpoints of the same run improved the measured trade-off, and monotonically. On the same 53 frames at the same budget, with the same hook count, the run reads 21.5%, 22.8%, 24.0% and 25.8% at epochs 0 to 3, a gain of 4.3 points over the checkpoint this paper reports, and it was still training when we stopped measuring. We report the pinned checkpoint because every table in the paper has to come from one set of weights, and we do not read 4.3 points as a bound on what a converged run would give.

Seven mechanisms are not in the system, and each was removed by a measurement rather than by taste. The table lists them with the number that ended each (Table 27). Five were built and dropped, one is an extension the measurement did not support, and the last is the explanation of the tiling penalty that we found does not predict it. We report them because the shape of what fails is the part of this result most likely to transfer: the border estimators say the seam is not an interpolation problem, the halo exchange says exactness and adaptivity are in tension, and the frozen trunk says the ladder has to be trained through.

what was built	the measurement that ended it
First-order border extrapolation	worse than replication at all three rates: 0.198, 0.286 and 0.384 dB against 0.071, 0.123 and 0.218
Fitted AR(1) border padding	ahead of replication by 0.021 dB at q63, and bought with wall clock that no file in results/ records
A learned deblocking filter	0.0008 dB even with a perfect gate, against 1.09% of a decode
Halo exchange between tiles	exact at uniform depth, and the saving falls from 25.9% to 4.2% at q32 and from 19.2% to 0.4% at q63
Freezing the trunk	drift exactly 0.000 dB, and the oracle ceiling collapses with it to 11.5% at q0, 0.4% at q32 and -1.0% at q63
A tile size chosen per resolution	worth -0.06 points on the set mean over three rates, with the training confound running in its favour
The affected-area fraction	wrong by 137 to 241%, against 13 to 22% for a power law in the per-tile block count

Table 27. Seven mechanisms and what settled each. None is in the reported system. The supplement gives the full measurement behind every row.

A single decoder. All results are on DCVC-UF's intra decoder. Nothing in the method looks specific to it, since the ladder needs only a residual trunk with a shared head, but we have not measured a second decoder to check.

8. Conclusion

The intra decoder of DCVC-UF can be run at a depth chosen per tile from what that tile contains. At a 0.1 dB budget this removes 29.2% to 15.2% of its multiply-accumulates for a BD-Rate cost of 1.03%, with no change to the encoder and none to the coded payload.

Three lessons we would expect to carry to decoders of this class, a residual trunk behind a shared head, and none of them is about early exit.

Tiling is the dominant cost and it is governed by depth. All of that cost traces back to a single 3×3 that is 0.29% of the arithmetic, and the penalty grows as the square of how many such convolutions run per tile. The affected-area fraction usually quoted alongside it predicts that penalty badly.

The exact remedy is in tension with the thing it enables. Giving each convolution its real neighbour is bit-identical at uniform depth, and under routing it destroys the allocation, because routing is the deliberate violation of the condition that makes it exact. We expect any method that combines spatial adaptivity with tiled inference to walk into this.

A distortion budget is only a control variable inside a measurable band. Below the floor the budget admits nothing at all, and above saturation more of it buys nothing (Figure 26). A saving quoted without saying where in that band it sits has left out the part of the result a reader most needs.

We would attach three cautions to the measurements themselves. A saving in operations is an optimistic bound on a saving in time, and the optimism scales with the saving. A learned router should be compared against a free one. Routing on the bits already spent per tile needs no parameters and no training, it adds nothing to the stream, and it beats our trained head at every rate we measured, while agreeing with the Lagrangian oracle on fewer tiles than the head does. We read that as a warning about the metric quite as much as about the head. Finally, a timing harness will report numbers whether or not it is timing the right device. Ours timed the wrong one for months, and what gave it away was a decoder that appeared to slow down with the quality index.

References

[1] S. Wang et al. Adaptive patch exiting for scalable single image super-resolution. ECCV, 2022.

[2] J. Ballé et al. Variational image compression with a scale hyperprior. ICLR, 2018.

[3] G. Huang et al. Multi-scale dense networks for resource efficient image classification. ICLR, 2018.

[4] G. Bjøntegaard. Calculation of average PSNR differences between RD curves. VCEG-M33, 2001.

[5] B. Bross et al. Overview of the Versatile Video Coding standard. IEEE TCSVT, 2021.

[6] Z. Cheng et al. Learned image compression with discretized Gaussian mixture likelihoods. CVPR, 2020.

[7] W. Fedus et al. Switch Transformers. JMLR, 2022.

[8] G. Huang et al. Glance and Focus networks for dynamic visual recognition. IEEE TPAMI, 2023.

[9] G. J. Sullivan et al. Overview of the High Efficiency Video Coding standard. IEEE TCSVT, 2012.

[10] E. Jang et al. Categorical reparameterization with Gumbel-softmax. ICLR, 2017.

[11] T. Kaseva et al. Per-channel autoregressive linear prediction padding in tiled CNN processing of 2D spatial data. arXiv:2502.12300, 2025.

[12] X. Kong et al. ClassSR: a general framework to accelerate super-resolution networks by data characteristic. CVPR, 2021.

[13] J. Li, B. Li, Y. Lu. Neural video compression with feature modulation. CVPR, 2024.

[14] J. Li, B. Li, Y. Lu. Uniformly accelerated neural video codec with feature refinement. ACM MM, 2024.

[15] Y. Kaya et al. Shallow-deep networks: understanding and mitigating network overthinking. ICML, 2019.

[16] L. Wang et al. Auxiliary-loss-free load balancing strategy for mixture-of-experts. arXiv:2408.15664, 2024.

[17] M. Phuong, C. H. Lampert. Distillation-based training for multi-exit architectures. ICCV, 2019.

[18] S. Scardapane et al. Why should we add early exits to neural networks? Cognitive Computation, 2020.

[19] W. Sun et al. Multi-exit self-distillation with appropriate teachers. FITEE, 2024.

[20] S. Teerapittayanon et al. BranchyNet: fast inference via early exiting from deep neural networks. ICPR, 2016.

[21] B. Bross et al. Versatile Video Coding. IEEE TCSVT, 2021.

[22] F. Yang et al. Slimmable compressive autoencoders for practical neural image compression. CVPR, 2021.

[23] M. A. Yilmaz et al. Slimmable video codec. CVPRW, 2022.

[24] A. Kuznetsova et al. The Open Images Dataset V4. IJCV, 2020.

[25] A. Mercat et al. UVG dataset: 50/120fps 4K sequences for video codec analysis. ACM MMSys, 2020.

[26] H. Wang et al. MCL-JCV: a JND-based H.264/AVC video quality assessment dataset. ICIP, 2016.

[27] T. Blard et al. Spatial competition for low-complexity learned image compression. arXiv:2605.13243, 2026.

[28] Y. Shoham, A. Gersho. Efficient bit allocation for an arbitrary set of quantizers. IEEE TASSP, 1988.

[29] A. Ortega, K. Ramchandran. Rate-distortion methods for image and video compression. IEEE SPM, 1998.

[30] Adaptive test-time compute allocation for reasoning LLMs via constrained policy optimization. arXiv:2604.14853, 2026.

[31] H. A. Alhaija et al. Local padding in patch-based GANs for seamless infinite-sized texture synthesis. arXiv:2309.02340, 2023.

[32] C. Innamorati et al. Overlap training to mitigate inconsistencies caused by image tiling in CNNs. arXiv:1812.02203, 2018.

[33] Z. Jia et al. Towards practical real-time neural video compression. CVPR, 2025.

[34] B. A. Hasan et al. Adaptive inference: theoretical limits and unexplored opportunities. arXiv:2402.04359, 2024.

[35] Y. Gao et al. Exploring the rate-distortion-complexity optimization in neural image compression. CVIU, 2024.

[36] Y.-H. Ho et al. On the rate-distortion-complexity trade-offs of neural video coding. arXiv:2410.03898, 2024.

[37] C. Zhang, W. Gao. Learned rate control for frame-level adaptive neural video compression via dynamic neural network. arXiv:2508.20709, 2025.

[38] Y. Geifman, R. El-Yaniv. Selective classification for deep neural networks. NeurIPS, 2017.

[39] D. Madras et al. Predict responsibly: improving fairness and accuracy by learning to defer. NeurIPS, 2018.

[40] H. Mozannar, D. Sontag. Consistent estimators for learning to defer to an expert. ICML, 2020.

[41] G. DeSalvo et al. Budgeted multiple-expert deferral. arXiv:2510.26706, 2025.

[42] M. Dong, M. Lu, and Z. Ma. Accelerating block-level rate control for learned image compression. arXiv:2409.01009, 2024.

[43] Y. Li et al. Learned image compression with hierarchical progressive context modeling. ICCV, 2025.

[44] D. He, Z. Yang, W. Peng, R. Ma, H. Qin, Y. Wang. ELIC: efficient learned image compression with unevenly grouped space-channel contextual adaptive coding. CVPR, 2022.

[45] R. Zou, C. Song, Z. Zhang. The devil is in the details: window-based attention for image compression. CVPR, 2022.

[46] J. Liu, H. Sun, J. Katto. Learned image compression with mixed transformer-CNN architectures. CVPR, 2023.

[47] W. Jiang, R. Wang. MLIC++: linear complexity multi-reference entropy modeling for learned image compression. ICML Neural Compression Workshop, 2023.

[48] H. Li, S. Li, W. Dai, C. Li, J. Zou, H. Xiong. Frequency-aware transformer for learned image compression. ICLR, 2024.

[49] S. Qin et al. MambaVC: learned visual compression with selective state spaces. arXiv:2405.15413, 2024.

[50] H. Fu, J. Liang, Z. Fang, J. Han, F. Liang, G. Zhang. WeConvene: learned image compression with wavelet-domain convolution and entropy model. ECCV, 2024.

[51] D. Minnen, S. Singh. Channel-wise autoregressive entropy models for learned image compression. ICIP, 2020.

[52] J. Li, B. Li, Y. Lu. Neural video compression with diverse contexts. CVPR, 2023.

[53] X. Huang, S. Belongie. Arbitrary style transfer in real-time with adaptive instance normalization. ICCV, 2017.

[54] M. Wolczyk et al. Zero time waste: recycling predictions in early exit neural networks. NeurIPS, 2021.

[55] L. Yang et al. Resolution adaptive networks for efficient inference. CVPR, 2020.